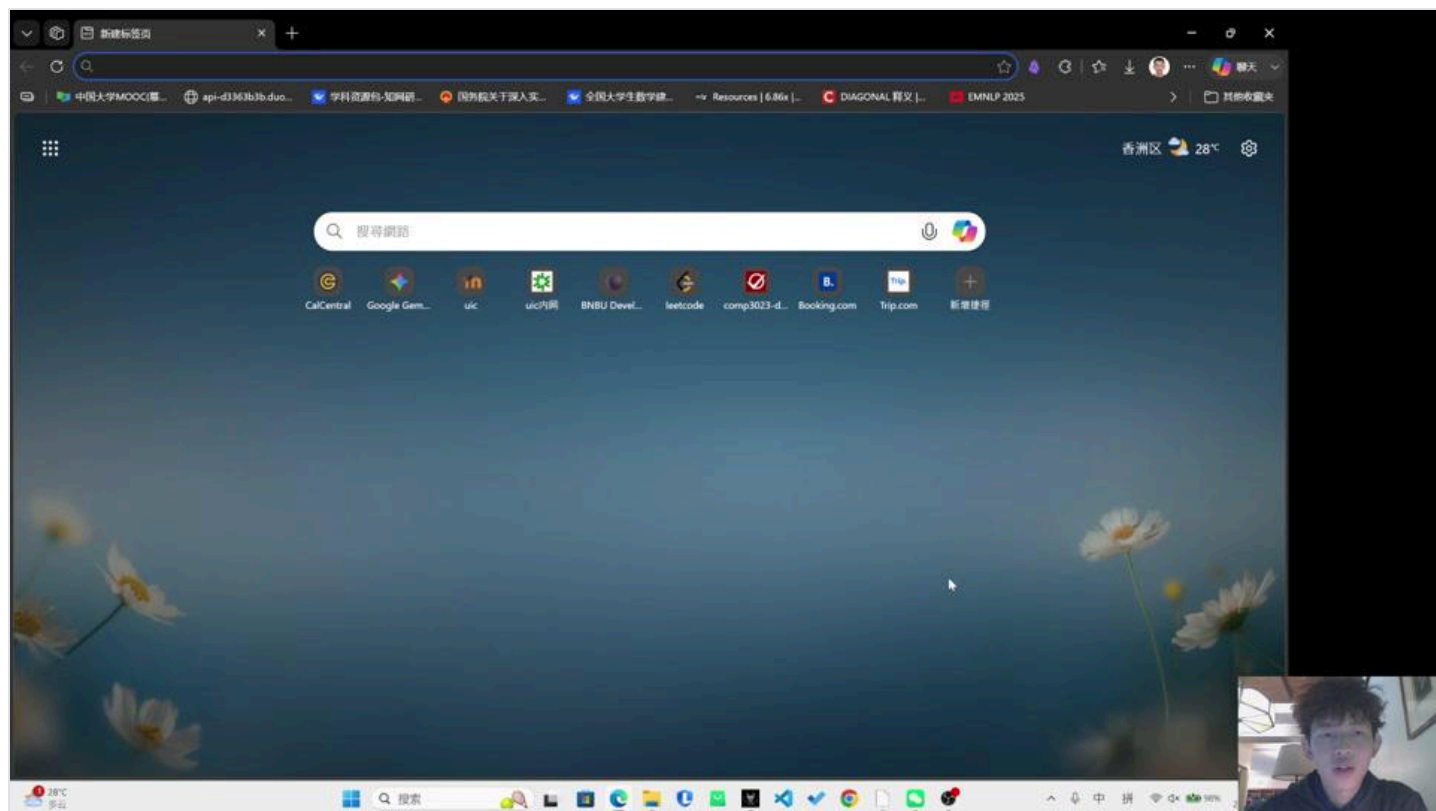


腾讯云宝塔面板保姆级教程 — 从零到全栈部

教学视频：



以 BNU Sports 校园体育平台为例，手把手教你理解服务器、网站、数据库、前端、后端之间的所有关系。

⚠ 严禁触碰 — 生产环境保护区

> 以下两个站点的任何东西，绝对不要修改、删除、或覆盖：

🚫 禁区	端口	包含什么	为什么不能碰
BNBU Sports 体育平台	:96	网站文件、Nginx 配置、数据库 123_207_5_70_96 、 Node.js :3001	全校师生正在使用，任何误操作会导致系统宕机
BNBU Developer 知识经验库	:100	网站文件、Nginx 配置、数据库 video_showcase 、 Node.js :3002	开发者知识库正在运行，包含所有部署文档和历史经验

具体禁止事项：

代码块

1

🚫 禁止在宝塔「网站」 → 123.207.5.70:96 或 123.207.5.70:100 → 点任何按钮

2

🚫 禁止在宝塔「文件」 → /www/wwwroot/123.207.5.70_96/ 或 .../123.207.5.70_100/ 修改/删除任何文件

3

🚫 禁止在宝塔「文件」 → /www/wwwroot/bnbu-api-v1/ 或 .../video-showcase-api/ 修改/删除任何文件

4

🚫 禁止在宝塔「数据库」 → 123_207_5_70_96 或 video_showcase → 执行任何 SQL

5

🚫 禁止在宝塔「终端」 → 执行 pm2 restart bnbu-api-v1 / pm2 restart vshow-api / pm2 delete ...

6

🚫 禁止在宝塔「终端」 → 执行任何 DROP、DELETE、ALTER 等数据库命令（除非你明确知道自己在做什么）

7

🚫 禁止在宝塔「网站」 → 修改 Nginx 配置后直接保存重载（先在别处测试！）

> 记住：看可以，点可以，读可以。改不行，删不行，执行不行。

本教程的所有动手操作，都只在你自己新建的端口上进行（见实践篇），不会碰这两个生产站点。

- [🚫 严禁触碰 — 生产环境保护区]
- [序章：为什么需要服务器？]
- [第一章：整体架构 — 请求是如何穿越整条链路的]
 - [1.3 端口 — 服务器这座大楼的每一扇门]
- [第二章：认识宝塔面板四大金刚]
- [第三章：前端静态文件 — Nginx 的饭碗]
- [第四章：后端 API 文件 — Node.js 的饭碗]
- [第五章：Nginx 反向代理 — 拴住前后端的绳子]
- [第六章：数据库 — 一切数据的家]
- [第七章：Web 端如何连接后端]
- [第八章：Android App 如何连接后端]
- [第九章：iOS App 如何连接后端]
- [第十章：完整上传部署流程]
- [第十一章：日常维护与故障排查]
- [📝 实践篇：自己动手搭建一个网站（安全区域）]

序章：为什么需要服务器？

用开餐馆来理解整件事

想象你要开一家中餐馆：

开餐馆的环节	对应服务器端	通俗解释
店面	腾讯云服务器（一台虚拟电脑）	你租了个铺面
门牌号	IP 地址 123.207.5.70	客人知道地址才能来

大堂桌子	Nginx（前端静态文件服务）	客人进来就看到菜单、坐下
厨房	Node.js（后端 API）	客人点菜后，厨房做菜
传菜口	Nginx 反向代理 <code>/api/*</code>	服务员把菜单递给厨房
冰箱/仓库	MySQL 数据库	食材储存的地方
菜单	<code>index.html</code> + <code>app.js</code> + <code>styles.css</code>	前端页面的长相和交互
菜谱	<code>server.js</code> （后端代码）	厨房怎么做菜的逻辑
大堂经理	宝塔面板（Baota）	你在吧台总控一切

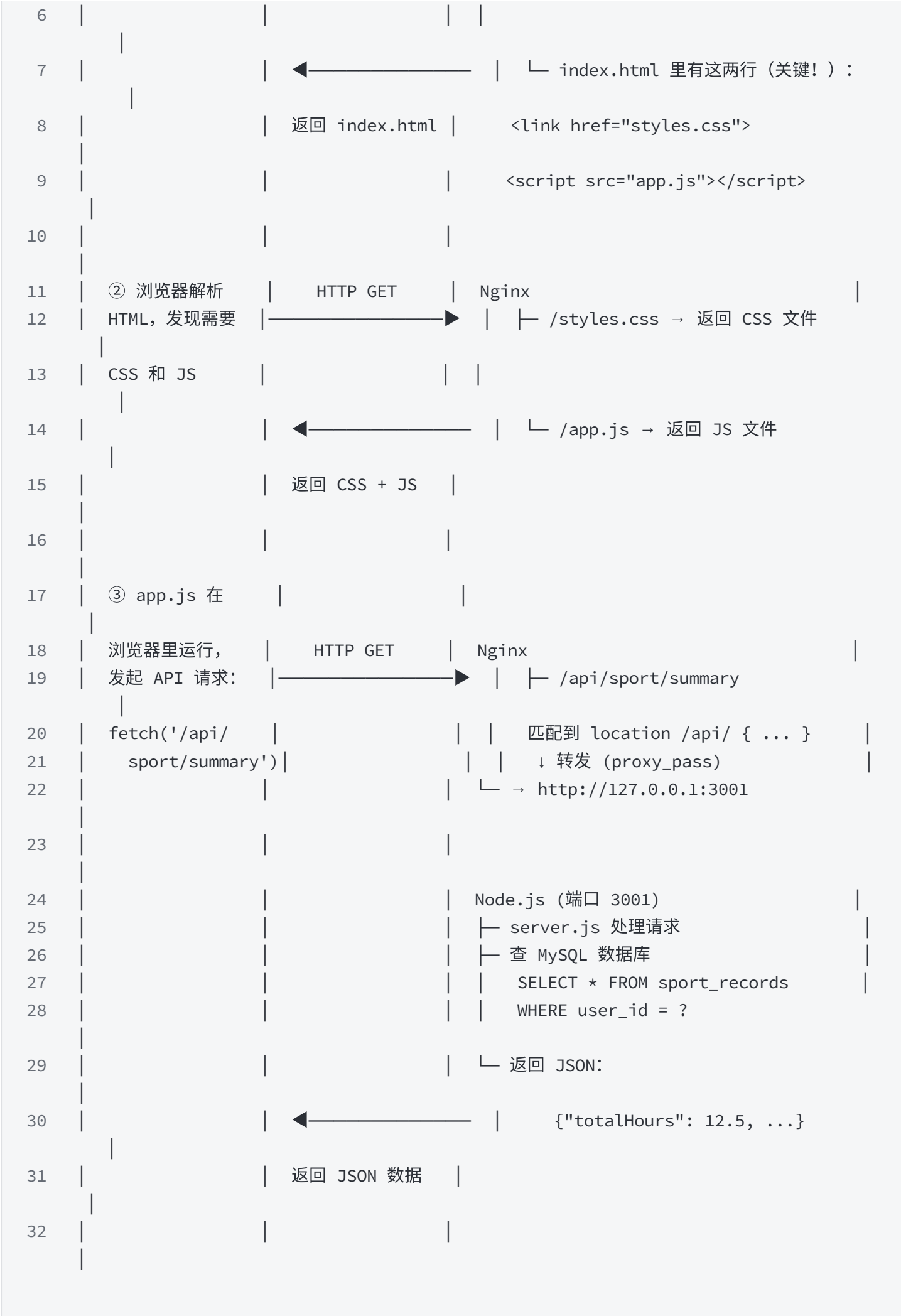
关键道理：客人（用户）打开浏览器或 App，其实是在远程访问一台远在腾讯云机房的电脑。这台电脑 24 小时开机，等着任何人来访问。

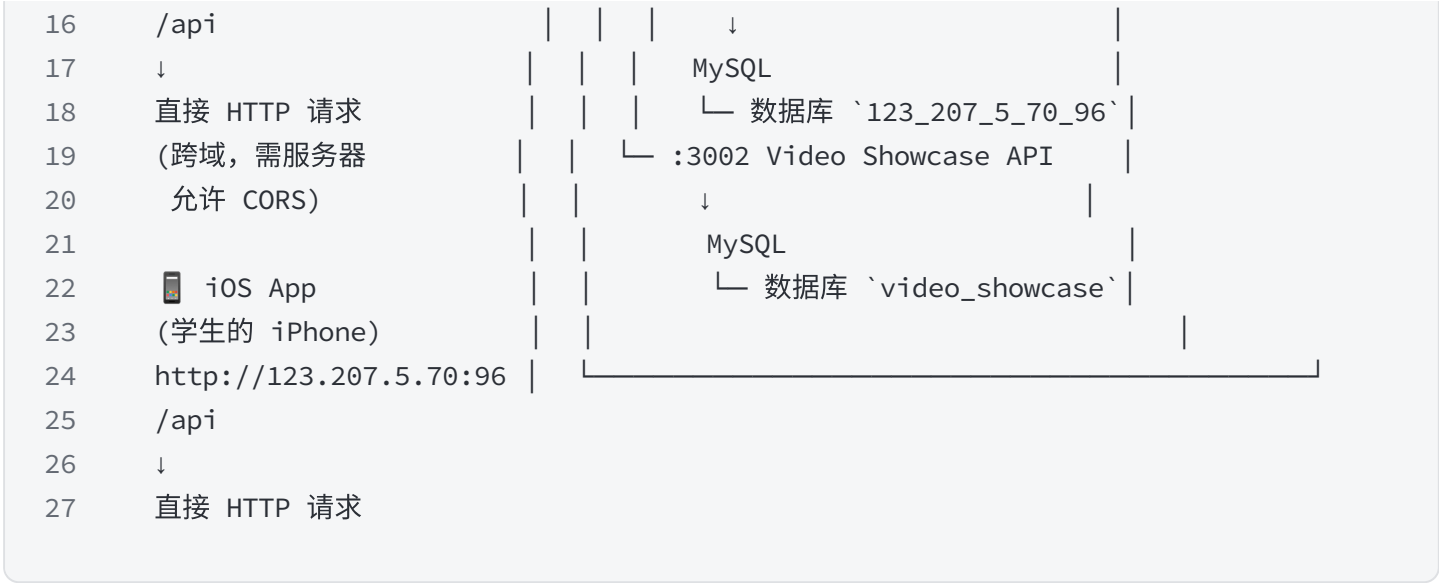
第一章：整体架构 — 请求是如何穿越整条链路的

1.1 一个真实的请求之旅

假设小明打开浏览器，输入 `http://123.207.5.70:96`，点击"查询我的体育学时"按钮。下面以 **BNBU Sports 的真实项目** 为例，一步步追踪发生了什么：







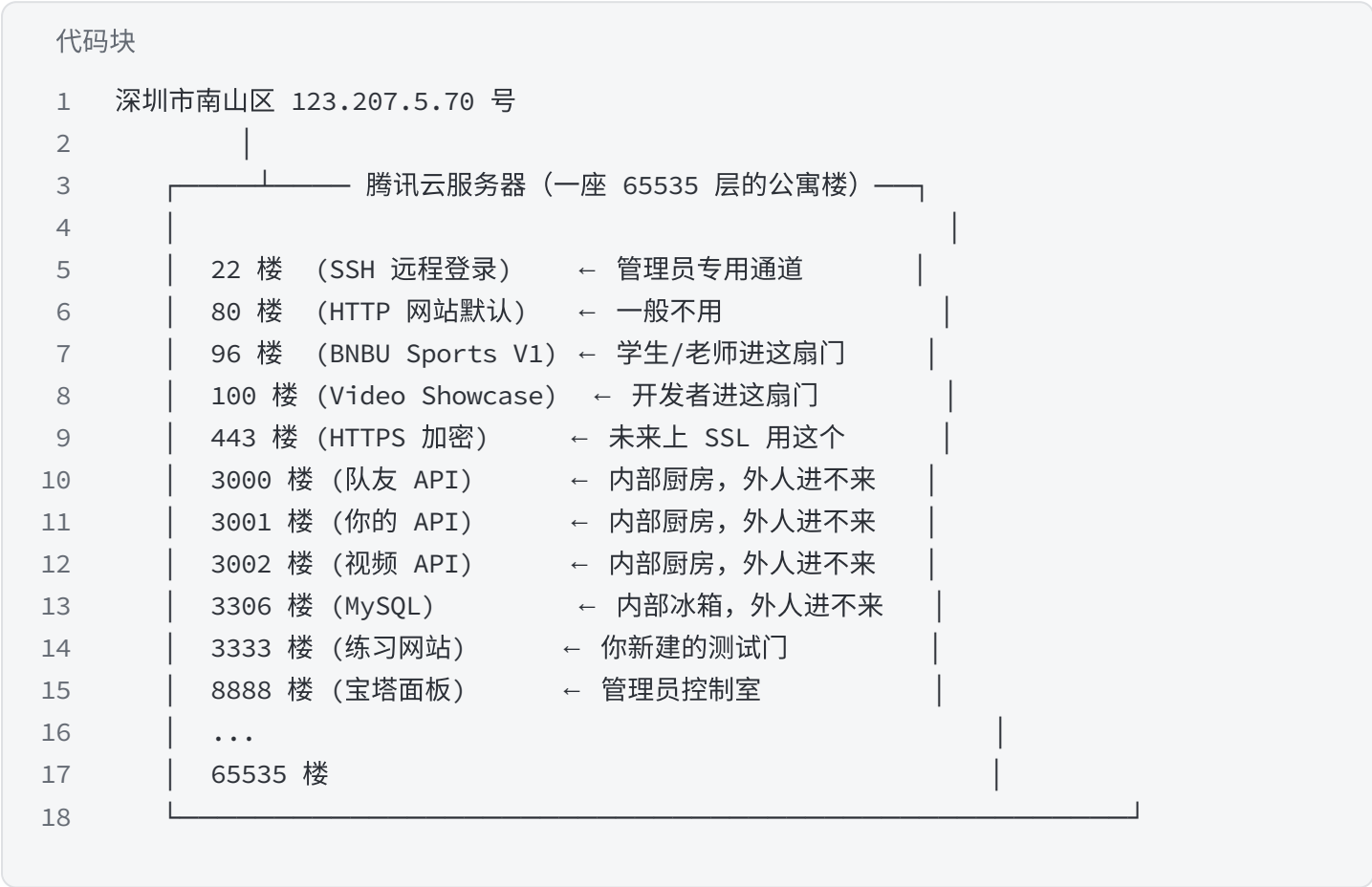
三端的区别：

特性	Web 端（浏览器）	Android App	iOS App
请求方式	<code>fetch('/api/...')</code> 相对路径	OkHttp <code>http://123.207.5.70:96/api/...</code> 绝对路径	URLSession <code>http://123.207.5.70:96/api/...</code> 绝对路径
是否有跨域问题	❌ 没有（同源）	✅ 有（App 不是浏览器，但需要服务器设置 CORS）	✅ 有
谁在发请求	浏览器的 JavaScript 引擎	Android 的 OkHttp 网络库	iOS 的 URLSession 网络库
API 地址在哪配置	<code>app.js</code> 第 51 行 <code>apiBaseUrl: ""</code>	<code>StudentApiClient.kt</code> 第 171 行 <code>DefaultBaseUrl</code>	<code>StudentAPIClient.swift</code> 第 7 行 <code>baseUrl</code>
好处	不用配地址，自动跟当前域名一致	可以离线使用（MVP 目前用本地 Mock 数据）	同左

1.3 端口 — 服务器这座大楼的每一扇门

端口是整条链路中最容易被忽略、但又最重要的概念。不理解端口，就没办法理解为什么有 :96、:3001、:3306 这些数字。

用一个公寓楼来理解



一个 IP 地址只有一个（ 123.207.5.70 ），但可以开 65535 扇门（端口）。每扇门后面可以有一个完全不同的服务。

端口是怎么工作的？

当你打开浏览器输入 <http://123.207.5.70:96> 时：



4 ↓
5 互联网路由：包到了腾讯云机房 → 找到 123.207.5.70 这台机器
6 ↓
7 服务器操作系统：有人敲门！96 号门... 让我查一下谁在监听 96 号门
8 ↓
9 Nginx：是我！我在 96 号门等着呢。包给我，我来处理。

关键理解：如果你在浏览器输入 `http://123.207.5.70:3001`，服务器操作系统会去找 3001 号门，发现 Node.js 在监听。但如果防火墙没开 3001 端口，这个请求在到达服务器之前就被丢弃了。

端口的两大分类

代码块

1				
2	端口		65535	
3				
4	1 - 1023	1024 - 65535		
5	系统保留端口	用户可用端口		
6	(需要 root 权限)	(普通程序就能用)		
7				
8	22 SSH	3000 队友 API (Node.js)		
9	80 HTTP 默认	3001 你的 API (Node.js)		
10	443 HTTPS 默认	3002 视频 API (Node.js)		
11	3306 MySQL 默认	3333 练习网站 (Nginx)		
12	8888 宝塔面板	3334 练习 API (Node.js)		
13		8080 本地开发常用		
14				

端口不对外开放 = 门被封死了（这是好事！）

这是最重要的安全概念。在服务器上：

端口	对外开放？	谁能访问	为什么

96	✔ 是	全世界	学生和老师需要访问网页
100	✔ 是	全世界	开发者需要访问知识库
8888	✔ 是	只有你知道密码	管理员需要管理服务器
3001	✗ 否	只有服务器自己	API 不应直接暴露，通过 Nginx 转发
3002	✗ 否	只有服务器自己	同上
3306	✗ 否	只有服务器自己	数据库绝不能直接暴露在公网
22	⚠ 部分	特定 IP 白名单或密钥登录	SSH 远程管理

这个安全模型是怎么实现的？

- 1. 防火墙（腾讯云安全组）：在腾讯云控制台配置，决定哪些端口允许外网流量进入
- 2. Nginx 绑定：Node.js 启动时写 `app.listen(3001, '127.0.0.1')` —— 只监听本地回环地址，外网请求到不了
- 3. 宝塔面板安全设置：宝塔 → 安全 → 防火墙，可以细粒度控制端口

代码块

```
1 # 在服务器终端验证
2 netstat -tlnp | grep LISTEN
3 # 输出示例：
4 # tcp      0      0 0.0.0.0:96          0.0.0.0:*    LISTEN  nginx
5 # tcp      0      0 0.0.0.0:100         0.0.0.0:*    LISTEN  nginx
6 # tcp      0      0 0.0.0.0:8888        0.0.0.0:*    LISTEN  python (baota)
7 # tcp      0      0 127.0.0.1:3001      0.0.0.0:*    LISTEN  node
8 # tcp      0      0 127.0.0.1:3002      0.0.0.0:*    LISTEN  node
9 # tcp      0      0 127.0.0.1:3306      0.0.0.0:*    LISTEN  mysqld
10 #
11 #      ↑              ↑
12 #      监听地址      0.0.0.0 = 监听所有网卡（外网能访问）
13 #      127.0.0.1 = 只监听本地回环（外网完全访问不到）
14 #      0.0.0.0 = 监听所有网卡/所有IP（公网可访问）
```

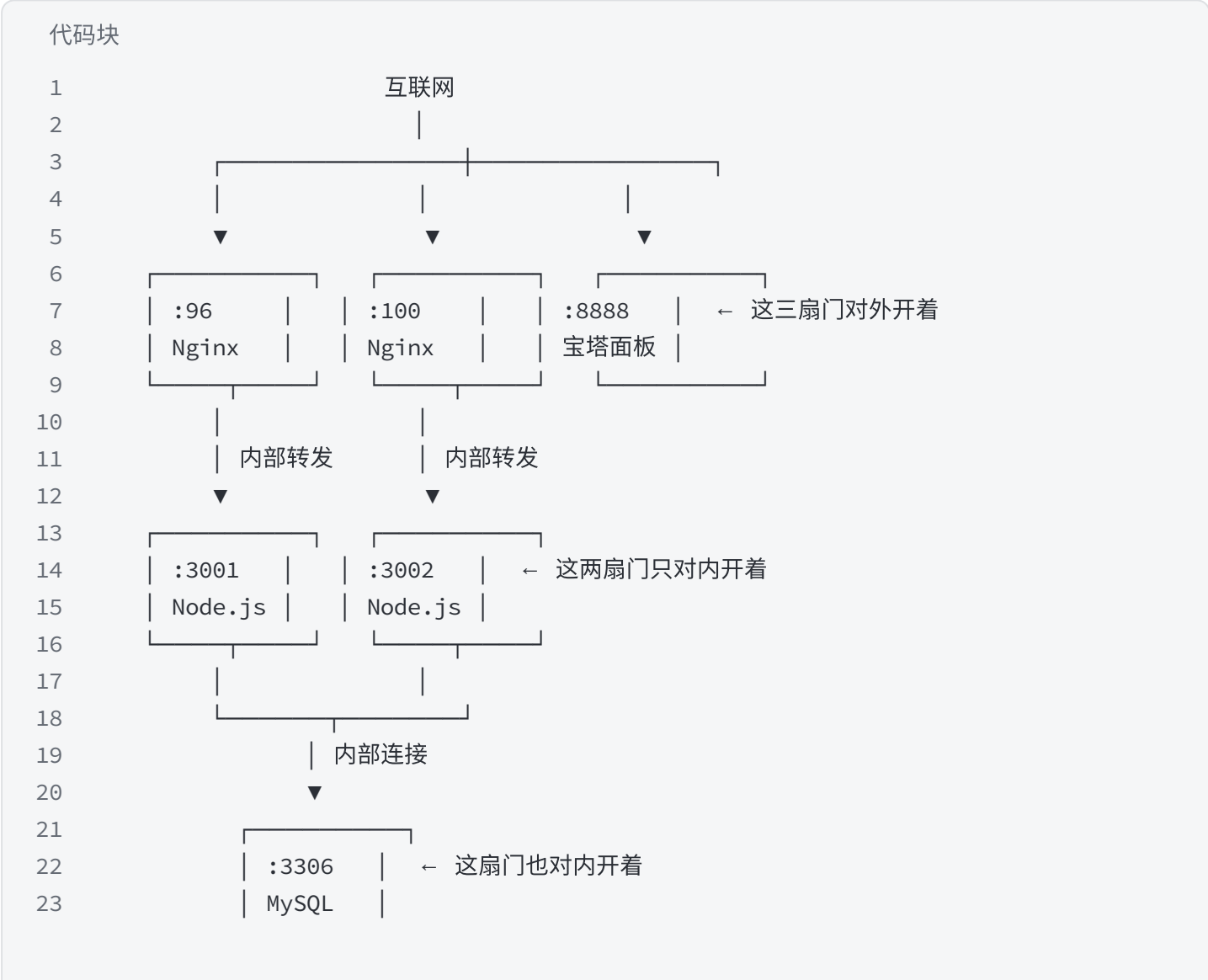
看懂这一行：

代码块

```
1  tcp  0  0  127.0.0.1:3001  0.0.0.0:*  LISTEN  node
2              ↑              ↑
3          只监听 127.0.0.1    任何来源 IP 都无法连接（除非来自本机）
```

127.0.0.1:3001 前面的 127.0.0.1 意思是：只有从服务器自己发出的请求才能到达 3001 端口。外界用户输入 <http://123.207.5.70:3001> 是绝对访问不到的。只有 Nginx（它也在本机上）能通过 <http://127.0.0.1:3001> 跟 Node.js 说话。

BNBU 项目的端口全景图



怎么检查一个端口是否开放？

代码块

```
1  # 方法一：在服务器上检查谁在监听
2  netstat -tlnp | grep 3001
3
4  # 方法二：在服务器上本地测试
5  curl http://127.0.0.1:3001/api/health
6
7  # 方法三：从你的电脑测试（测试外网是否可达）
8  # PowerShell:
9  Test-NetConnection 123.207.5.70 -Port 96
10 # 或浏览器直接打开:
11 http://123.207.5.70:96/api/health # 能访问说明 Nginx 和 API 都通
12 http://123.207.5.70:3001         # 应该访问不到（端口被封）
```

宝塔面板如何管理端口

代码块

```
1  宝塔 → 安全 → 防火墙
2  |—— 查看所有已开放的端口
3  |—— 添加端口规则（开放新端口）
4  |—— 删除端口规则（封掉端口）
5  |—— 注意：这里是宝塔层面的防火墙
6          腾讯云控制台还有一层「安全组」防火墙
7          两层都要放行，外网才能访问
```

常见坑：你在宝塔开了 3333 端口，网站也建好了，但外网访问不了 → 大概率是**腾讯云安全组没有放行 3333 端口**。去腾讯云控制台 → 轻量应用服务器 → 防火墙 → 添加规则。

端口号的命名习惯（记住这组数字）

你能自己选的端口范围	推荐
1024 - 65535	随使用
常用习惯	
3000-3999	Node.js API 服务
8000-8999	开发/测试
80, 443	HTTP/HTTPS 默认（别乱占）
3306	MySQL 默认（别乱占）
22	SSH 默认（别乱占）

一句话总结： IP 地址是服务器在互联网上的门牌号，端口是这栋楼里不同房间的门牌号。同一个 IP，不同端口，看到的是完全不同的服务。

第二章：认识宝塔面板四大金刚

宝塔面板就是一个**网页版服务器管理工具**。你不用记 Linux 命令，点鼠标就能管理整台服务器。

2.1 登录宝塔面板

你的服务器是腾讯云轻量应用服务器，IP 是 123.207.5.70。宝塔面板端口默认是 8888：

代码块

- 1 浏览器打开：http://123.207.5.70:8888
- 2 用户名：通常是 admin 或安装时设置的
- 3 密码：安装时设置的

2.2 四大金刚之首：网站（Website）

这相当于餐馆的大堂经理。进去后你会看到：

代码块

```
1  | _____|
2  |  宝塔面板 → 网站                |
3  | _____|
4  |                |
5  |  站点列表：                |
6  |  | _____| |
7  |  | 123.207.5.70:96    BNB Sports V1      | |
8  |  | 状态：运行中      Nginx 反向代理已配置  | |
9  |  | [设置] [配置文件] [日志] [重载]        | |
10 |  | _____| |
11 |  | _____| |
12 |  | 123.207.5.70:100   视频展示平台        | |
13 |  | 状态：运行中      Nginx 反向代理已配置  | |
14 |  | [设置] [配置文件] [日志] [重载]        | |
15 |  | _____| |
16 |                |
17 | _____|
```

这里每个"站点"就是一个**网站的入口**。它可以做两件事：

事情一：提供静态文件（前端）

当浏览器请求 `http://123.207.5.70:96/` 时，Nginx 会去**网站根目录**找文件。

把 BNB Sports 前端文件放在：

代码块

```
1  /www/wwwroot/123.207.5.70_96/
```

2	— index.html	← 网站入口 (HTML 骨架)
3	— app.js	← 所有业务逻辑 (JS 大脑)
4	— styles-campus-blue.css	← 样式表之一
5	— styles.css	← 样式表之二

你只需要把文件上传到这个目录，网站就能被访问到。

事情二：反向代理（连接后端）

在同一个站点配置里，还有一段 Nginx 配置，告诉它：收到 `/api/` 开头的请求时，别从文件里找，转发给后端的 **Node.js** 进程。

点站点旁边的「配置文件」，你会看到类似这样的配置：

代码块

```

1  server {
2      listen 96;                # 这个站点监听 96 端口
3      root /www/wwwroot/123.207.5.70_96; # 静态文件从这里读
4      index index.html;
5
6      # — 这一段处理普通网页请求 —
7      location / {
8          try_files $uri $uri/ /index.html; # 找不到文件就交给 index.html
9      }
10
11     # — 这一段处理 API 请求（关键！） —
12     location /api/ {
13         proxy_pass http://127.0.0.1:3001; # 转发给 Node.js 的 3001 端口
14         proxy_set_header Host $host;
15         proxy_set_header X-Real-IP $remote_addr;
16     }
17 }
```

> ★ Insight

>

> `location /api/` 是整个架构的"接线端子"。没有它，前端 `fetch('/api/sport/summary')` 发出去请求就不知道该找谁。

>

> `127.0.0.1:3001` 只在服务器内部有意义。外界用户直接访问 `http://123.207.5.70:3001` 是访问不到的（也不应该让外界直接访问），必须通过 Nginx 的 `/api/` 这条安全通道。

>

> 一条原则：外部只暴露 Nginx 端口（96、100），Node.js 端口（3001、3002）关在防火墙里。

>

2.3 四大金刚之二：数据库（Database）

这相当于餐馆的冰箱和仓库。

宝塔面板 → 数据库，你会看到：

代码块

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

宝塔面板 → 数据库

数据库列表：

123_207_5_70_96MySQL大小：2MB

用户名：123_207_5_70_96

密码：Bd84EKfpw3XSmheB

[管理][备份][权限]

video_showcaseMySQL大小：1MB

用户名：video_showcase

密码：TnKjjwmX1skSY65H

[管理][备份][权限]

点「管理」按钮 → 进入 **phpMyAdmin**（数据库的可视化管理工具）。你会看到：

代码块

```
1  phpMyAdmin → 数据库 123_207_5_70_96
2  |—— semesters      ← 学期表
3  |—— users          ← 用户表（学生、老师、管理员）
4  |—— courses        ← 课程表
5  |—— sport_records  ← 打卡记录表（学生在 App 上提交的）
6  |—— reviews        ← 审核记录表
7  |—— student_progress ← 学生学时进度
8  |—— memberships   ← 组织成员表（校队、社团）
9  |—— notifications  ← 通知表
10 |—— exemptions     ← 免测申请表
11 |—— tasks          ← 课程任务表
12 |—— audit_logs     ← 操作审计日志
```

每张表是干什么的（通俗版）：

表名	通俗理解	谁往里写数据
users	花名册：所有学生、老师、管理员	管理员通过 Web 端导入
courses	课程总表：哪个老师教哪个班	管理员通过 Web 端配置
sport_records	打卡记录：小明某天跑了 2 公里	学生在 App 上提交打卡
reviews	审核记录：老师对小明的打卡说"通过"	老师在 Web 端审核
student_progress	学时进度：小明的课程学时和通用学时各完成了多少	系统自动计算
notifications	通知：告诉小明"你的打卡通过了"	系统自动生成
memberships	组织关系：小明是足球队的	管理员/Manager 配置

exemptions	免测申请：小明腿伤了不能跑 1000m	学生在 App 上提交申请
------------	---------------------	---------------

关键理解：数据库是唯一的数据真相来源（Single Source of Truth）。不管前端是 Web、Android 还是 iOS，所有数据都存到同一套 MySQL 表里。数据库不分三端。

2.4 四大金刚之三：终端（Terminal）

这相当于你直接走进餐馆的厨房操作间。

宝塔面板自带一个网页版终端（SSH），打开后就是服务器的命令行。你可以直接执行 Linux 命令而不需要额外安装 SSH 客户端。

高频使用的命令：

代码块

```
1  # ① 查看所有 Node.js 进程是否在运行
2  pm2 list
3  # 你应该看到：
4  # | bnbu-api-v1 | online | 3001 | # BNB Sports V1 API
5  # | bnbu-api | online | 3000 | # BNB Sports V2 API (队友的)
6  # | vshow-api | online | 3002 | # Video Showcase API
7
8  # ② 如果进程挂了（状态显示 stopped），重启它
9  pm2 restart bnbu-api-v1
10
11 # ③ 看日志（调试利器！）
12 pm2 logs bnbu-api-v1
13 # 输出最后 15 行日志，Ctrl+C 退出
14 # 如果前端调用 API 报错，一定来这看日志！
15
16 # ④ 检查端口是否被占用
17 netstat -tlnp | grep 3001
18 # 如果输出空行 → Node.js 没在跑
19 # 如果输出 tcp ... 127.0.0.1:3001 LISTEN → 正常运行
20
21 # ⑤ 验证 API 是否正常
```

```
22 curl http://127.0.0.1:3001/api/health
23 # 预期输出: {"ok":true,"service":"BNBU Sports API","db":"connected"}
24
25 # ⑥ 重载 Nginx (改了 Nginx 配置后必须执行)
26 nginx -s reload
27 # 或者: /etc/init.d/nginx reload
28
29 # ⑦ 安装 npm 依赖 (上传新的 package.json 后)
30 cd /www/wwwroot/bnbu-api-v1
31 npm install
```

> ★ Insight

>

> **宝塔终端有一个 PATH 坑：**宝塔的网页终端不会自动加载 `~/.bashrc`，所以 `node`、`npm`、`pm2` 可能报 `command not found`。解决办法：

>

> ```bash

> # 一次性创建软链接（重启也有效）

> ln -sf /root/.nvm/versions/node/v22.22.3/bin/node /usr/local/bin/node

> ln -sf /root/.nvm/versions/node/v22.22.3/bin/npm /usr/local/bin/npm

> ln -sf /root/.nvm/versions/node/v22.22.3/bin/pm2 /usr/local/bin/pm2

> ```

>

> 如果 NVM 版本号不对，先用 `find /root/.nvm -name "node" -type f` 找到实际路径。

>

2.5 四大金刚之四：文件（Files）

这相当于餐馆的储物架。所有文件都在这里管理 —— 上传、编辑、删除、改权限。

代码块

1 宝塔面板 → 文件

2 /



重要规律：

文件类型	放在哪里	谁负责提供它	改了之后要干嘛
前端 HTML/CSS/JS	/www/wwwroot/123.207.5.70_96/	Nginx 直接发送	什么也不用做（静态文件）
后端 server.js	/www/wwwroot/bnbu-api-v1/	Node.js 运行它	pm2 restart bnbu-api-v1
package.json	/www/wwwroot/bnbu-api-v1/	—	npm install + pm2 restart

静态文件（把文件直接发给浏览器）和 **API 文件**（运行逻辑代码）是两种完全不同的东西，放在**完全不同的目录里**。

第三章：前端静态文件 — Nginx 的饭碗

3.1 什么是"静态文件"？

静态文件就是**不需要服务器执行任何计算的纯文件**。就像你从书架上拿一本书直接看，书不需要"运行"。

代码块

```
1 index.html    → 浏览器直接解析 HTML 标签
2 app.js        → 浏览器直接执行 JavaScript 代码
3 styles.css    → 浏览器直接应用样式规则
4 logo.png      → 浏览器直接显示图片
```

3.2 Nginx 怎么 serve 静态文件？

Nginx 就是一个**文件转发员**。收到请求 → 找到文件 → 发回去。

代码块

```
1 浏览器：给我 /app.js
2 Nginx： 好的，我去 /www/wwwroot/123.207.5.70_96/app.js 拿一下 → 给你
3
4 浏览器：给我 /styles.css
5 Nginx： 好的，我去 /www/wwwroot/123.207.5.70_96/styles.css 拿一下 → 给你
```

但是，请求 `/api/sport/summary` 时，Nginx 不会去文件夹里找 `api/sport/summary` 这个文件。因为配置文件里写着：

```

1  location /api/ {
2      proxy_pass http://127.0.0.1:3001;    # ← 这个"proxy_pass"告诉 Nginx:
3  }                                          #    "别找文件了, 把请求转发出去!"

```

3.3 一个真实的例子：BNBU Sports 的 index.html

打开我们的 `index.html`（这是你在浏览器里看到的 HTML 骨架）：

代码块

```

1  <!DOCTYPE html>
2  <html lang="zh-CN">
3  <head>
4      <meta charset="UTF-8">
5      <title>BNBU Sports</title>
6      <link rel="stylesheet" href="styles.css">    <!-- ← 告诉浏览器去要 CSS -->
7  </head>
8  <body>
9      <div id="app"></div>                        <!-- ← 空的, 等 JS 来填充 -->
10     <script src="app.js"></script>              <!-- ← 告诉浏览器去要 JS -->
11 </body>
12 </html>

```

这个 HTML 非常小。页面上的所有内容（表格、按钮、表单）**都不是写在 HTML 里的**，而是由 `app.js` 在浏览器里动态生成的。这叫做 **SPA（Single Page Application，单页应用）**。

> ★ Insight

>

> **为什么用 SPA 模式？** 传统的网站每个页面都需要刷新，服务器每次返回一个新 HTML。SPA 的 `app.js` 像一个“舞台搭建工”，根据 URL 路径自己决定搭什么场景——不用刷新页面。

>

> 比如你从“打卡页面”切换到“成绩页面”，`app.js` 直接拆掉左边的场景、搭上右边的场景——速度快，体验流畅。

>

> **代价**：第一次加载需要下载完整的 `app.js`（可能很大），而且搜索引擎爬虫可能抓不到动态内容。但对于 BNB Sports 这种需要登录的内部系统，这完全不是问题。

>

3.4 缓存问题：改完文件为啥浏览器不更新？

这是所有前端开发者都会踩的坑。浏览器为了提高速度，会把 JS 和 CSS 文件**缓存**在用户电脑上。

代码块

- 1 第一次访问：下载 `styles.css` → 存到电脑缓存
- 2 第二次访问：嗯，这个文件我有了，直接用缓存的吧（不检查服务器有没有更新！）

解决办法：在 `index.html` 里版本号打戳。

代码块

- 1 `<!-- 每次修改 styles.css 或 app.js, 就把 ?v= 后面的日期改成今天 -->`
- 2 `<link rel="stylesheet" href="styles.css?v=20260629">`
- 3 `<script src="app.js?v=20260629"></script>`

浏览器看到 `styles.css?v=20260629` 和 `styles.css?v=20260628` 是**两个不同的 URL**，就会重新下载。

> **CLAUDE.md 的规定**：每次修改 `styles.css` 或 `app.js`，必须同时更新 `index.html` 里的 `?v=` 日期。忘记这一步 = 用户看到旧页面 = 部署失败。

第四章：后端 API 文件 — Node.js 的饭碗

4.1 什么是后端？它和前端有什么区别？

前端 = 在用户浏览器/手机里运行

后端 = 在服务器上运行

用 BNBU Sports 举例：

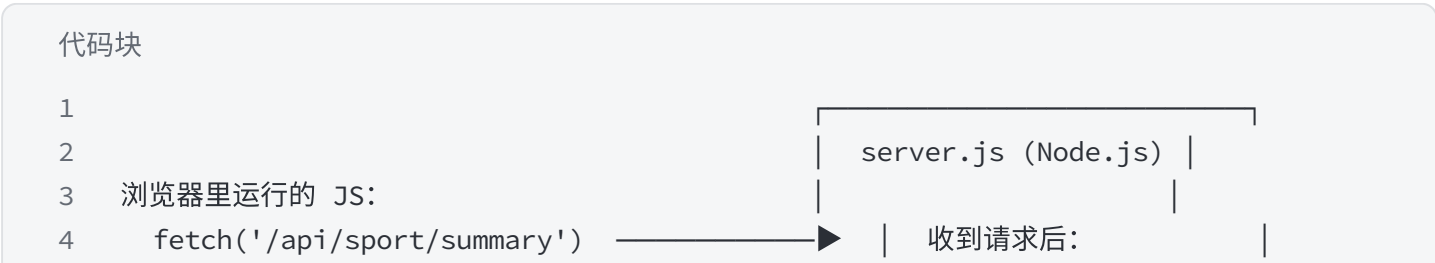
场景	前端做了什么	后端做了什么
学生提交打卡	显示表单、拍照、校验图片大小	接收数据、存到数据库、更新学时
老师审核	显示打卡列表、按钮、确认弹窗	读取记录、更新状态、发通知
看成绩	渲染成绩表格、计算百分比	查各科分数、算加权总分

为什么需要后端？

- 1. **网页无法直接连数据库**：如果 app.js 里写了 MySQL 密码，任何人都能在 F12 开发者工具里看到
- 2. **需要鉴权**：后端检查谁在请求，学生不能看老师的审核页面
- 3. **需要计算**：成绩加权、耐力跑换算这些需要跑在服务器上

4.2 Node.js 是干什么的？

Node.js 就是一个**能执行 JavaScript 文件的程序**。不只是浏览器能跑 JavaScript，服务器有了 Node.js 也能跑。



5	<code>alert('Hello')</code>	← 操作网页	① 从数据库查数据	
6	<code>console.log()</code>	← 调试	② 计算、校验	
7			③ 返回 JSON 结果	
8				
9			注意：它不能	
10			<code>alert()</code> 或操作 DOM	
11			因为它不在浏览器里！	
12				

同样的 JavaScript 语言，但运行环境不同，能做的事也不同。

4.3 后端 server.js 长什么样？（BNBU Sports 真实代码）

代码块

```

1  // 这是 /www/wwwroot/bnbu-api-v1/server.js 的精简版
2
3  // ① 引入库
4  const express = require('express');    // Web 框架
5  const mysql = require('mysql2');       // 数据库驱动
6
7  // ② 创建 Express 应用
8  const app = express();
9
10 // ③ 连接数据库
11 const db = mysql.createPool({
12   host: '127.0.0.1',
13   user: '123_207_5_70_96',
14   password: 'Bd84EKfpw3XSmheB',
15   database: '123_207_5_70_96',
16 });
17
18 // ④ 定义一个 API 端点 (= 一个 URL + 一个处理函数)
19 //   当有人 GET /api/health 时，执行这个大括号里的代码
20 app.get('/api/health', async (req, res) => {
21   // 测试数据库是否连着
22   const [rows] = await db.query('SELECT 1');
23   // 返回一个 JSON
24   res.json({ ok: true, db: rows.length > 0 });
25 });
26
27 // ⑤ 学生查询体育学时（关键业务接口）

```

```

28 app.get('/api/sport/summary', async (req, res) => {
29     const userId = req.query.userId; // 从请求里拿出 userId
30
31     // 去数据库查数据
32     const [records] = await db.query(
33         'SELECT credit_type, SUM(hours) as total FROM sport_records WHERE
34         user_id = ? GROUP BY credit_type',
35         [userId]
36     );
37     // 把结果返回给前端
38     res.json({
39         courseHours: records.find(r => r.credit_type === '课程')?.total || 0,
40         generalHours: records.find(r => r.credit_type === '通用')?.total || 0,
41     });
42 });
43
44 // ⑥ 启动服务, 监听 3001 端口
45 app.listen(3001, '127.0.0.1', () => {
46     console.log('BNBU Sports API running on http://127.0.0.1:3001');
47 });

```

逐行理解：

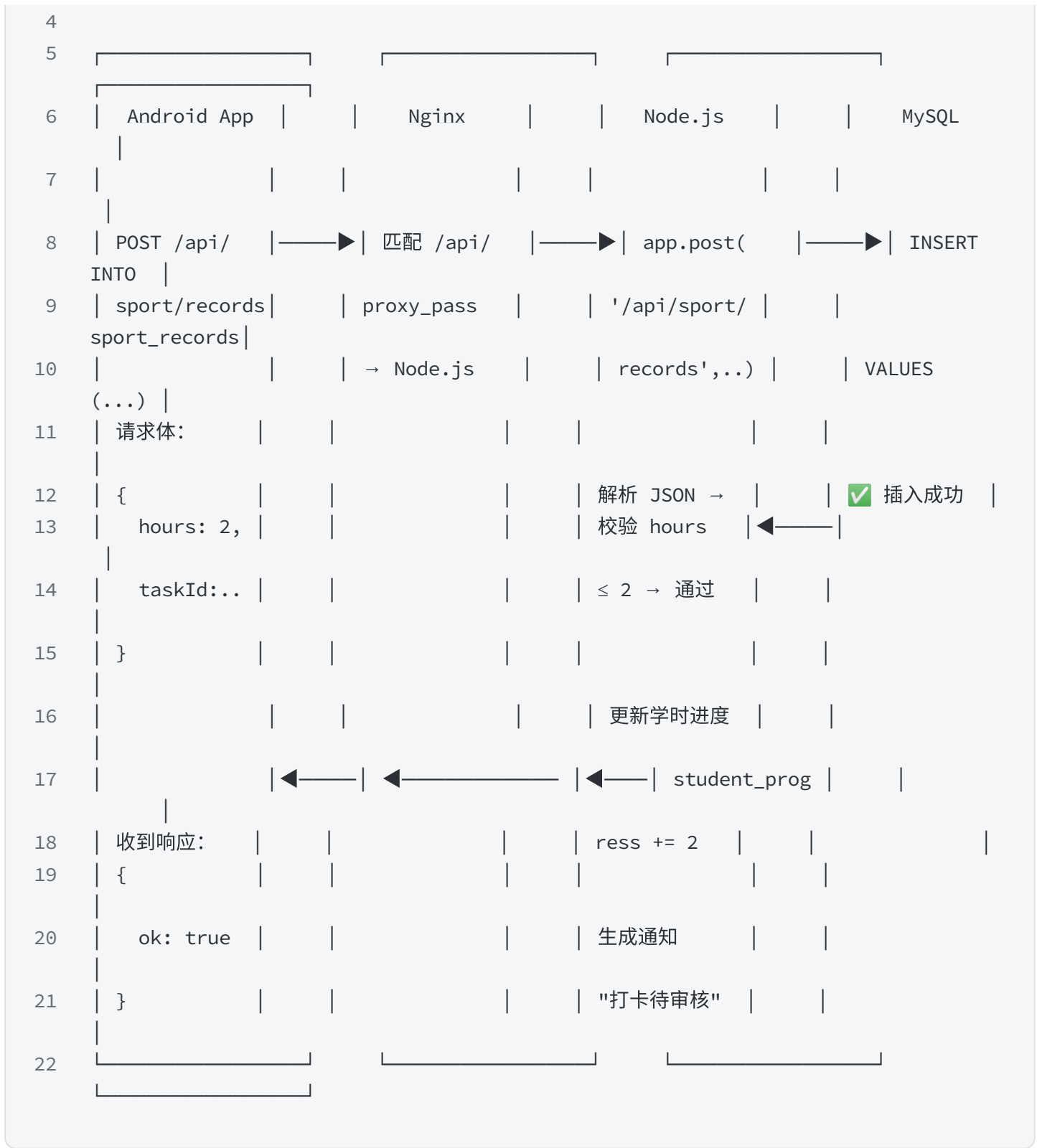
代码块

- 1 导入库 → 就像做菜前准备好调料和厨具
- 2 创建App → 支起一口锅
- 3 连接DB → 打开冰箱门, 随时取食材
- 4 定义端点 → 写菜谱: 客人点这个菜, 我就这么做
- 5 启动监听 → 系上围裙, 准备接客

4.4 一个 API 调用的完整数据流

代码块

- 1 学生 App → POST /api/sport/records → Nginx → Node.js → MySQL
- 2
- 3 详细展开:



第五章：Nginx 反向代理 — 拴住前后端的绳子

5.1 为什么需要反向代理？不能直接让前端调后端吗？

可以，但会很麻烦且有安全隐患：

代码块

```
1  ❌ 不好的方案：把 Node.js 端口暴露在外网
2
3  Android App → http://123.207.5.70:3001/api/sport/summary
4  Web 前端   → http://123.207.5.70:3001/api/sport/summary
5
6  问题：
7  1. Node.js 端口完全暴露，任何人都能扫描到它
8  2. Web 端：浏览器地址是 123.207.5.70:96，API 在 123.207.5.70:3001，
9     不同端口 = 跨域！浏览器会阻止请求（CORS 安全策略）
10 3. 外网流量直接打到 Node.js，没有 Nginx 的保护层
```

代码块

```
1  ✅ 好方案：Nginx 反向代理
2
3  对外只暴露 96 端口，/api/* 由 Nginx 转发到内部 3001
4
5  外界                Nginx (:96)                Node (:3001)
6  _____
7  GET /                → 返回 index.html
8  GET /app.js          → 返回 app.js
9  GET /api/sport/summary → 转发 → → → → → → 处理并返回 JSON
10 GET /api/health      → 转发 → → → → → → {"ok":true}
```

好处：

- 1. 同源：浏览器访问 123.207.5.70:96 ，API 也在 123.207.5.70:96/api/* —— 没有跨域问题
- 2. 安全：外界看不到 Node.js 的 3001 端口，只有 Nginx 能跟它说话
- 3. 性能：Nginx 处理静态文件非常快（不用经过 Node.js） ，同时可以缓存、限流、压缩

5.2 反向代理配置逐行解释

打开 BNBU Sports V1 的 Nginx 配置（宝塔 → 网站 → 123.207.5.70:96 → 配置文件），关键部分是：

代码块

```
1  # 块1: 处理静态文件请求
2  location / {
3      root /www/wwwroot/123.207.5.70_96;    # 网站根目录
4      index index.html;                      # 默认文件
5      try_files $uri $uri/ /index.html;      # 找不到文件就返回 index.html
6  }
7
8  # 块2: 处理 API 请求（反向代理的核心！）
9  location /api/ {
10     proxy_pass http://127.0.0.1:3001;      # 转发目标: 本机的 Node.js 3001 端口
11     proxy_http_version 1.1;
12     proxy_set_header Host $host;           # 把原始请求的域名传给 Node.js
13     proxy_set_header X-Real-IP $remote_addr; # 把用户真实 IP 传给 Node.js
14     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
15     proxy_set_header X-Forwarded-Proto $scheme;
16     proxy_read_timeout 30s;                # 等 30 秒没响应就超时
17     proxy_connect_timeout 5s;              # 连接超时 5 秒
18 }
```

逐行解释最重要的两句：

代码块

```
1  location /api/ {
```

→ **匹配规则**：URL 路径以 `/api/` 开头的请求，走这个块，不走上面的 `location /` 块。比如：

- `/api/health` → 走 `location /api/`
- `/api/sport/summary` → 走 `location /api/`
- `/app.js` → 走 `location /`（不匹配 `/api/`）
- `/index.html` → 走 `location /`

```
proxy_pass http://127.0.0.1:3001;
```

→ **转发指令**：把匹配到的请求原样转发给 `http://127.0.0.1:3001`，再把响应原样返回给用户。

转发过程：

代码块

```
1  用户请求: GET http://123.207.5.70:96/api/sport/summary?userId=42
2
3  Nginx 收到后, 发现路径是 /api/sport/summary?userId=42
4  匹配到 location /api/
5  执行 proxy_pass http://127.0.0.1:3001
6
7  相当于 Nginx 替用户去请求:
8  → GET http://127.0.0.1:3001/api/sport/summary?userId=42
9
10 Node.js 返回 JSON → Nginx 原样返回给用户
```

5.3 一个重要的对比：有代理 vs 无代理

代码块

```
1  用户浏览器输入 http://123.207.5.70:96
2  app.js 里写的是: fetch('/api/sport/summary')
3
4  浏览器自动拼接成: fetch('http://123.207.5.70:96/api/sport/summary')
5                                ↑           ↑
6                                域名和端口一致 = 同源! 没有跨域问题
7
8  如果 app.js 里写的是: fetch('http://123.207.5.70:3001/api/sport/summary')
9                                ↑           ↑
10                               域名一样但端口不同 = 跨域! 浏览器会拦截
```

>

> "同源策略"（Same-Origin Policy）是浏览器最核心的安全机制：只有协议 + 域名 + 端口完全一致的请求，浏览器才放行。端口不同也算跨域！

>

> 所以 Nginx 反向代理的一个隐藏好处是：它让 API 看起来和网页在**同一个端口**。浏览器不知道背后有 Node.js 在另一个端口监听 —— 它只看到 96 端口一视同仁地返回所有东西。

>

第六章：数据库 — 一切数据的家

6.1 MySQL 是什么？

MySQL 是一个**关系型数据库管理系统**。通俗说，它就是一个超级智能的 Excel：

普通 Excel	MySQL 数据库
一个 .xlsx 文件	一个 Database（数据库）
一个 Sheet 页签	一个 Table（表）
一行数据	一条 Record（记录）
一列	一个 Column（字段）
Ctrl+F 搜索	SELECT * FROM users WHERE name LIKE '%小明%'
只能一个人编辑	几百人同时读写不冲突

6.2 BNBU Sports 的数据表是怎么关联起来的

这是 BNBU Sports 最核心的几张表的关系：

代码块

```
1  users (用户表)
2  |— id: 1, name: 小明, role: student
3  |— id: 2, name: 张老师, role: teacher
4  |— id: 3, name: 刘管理员, role: admin
5
6  courses (课程表)
7  |— id: 101, name: 足球, teacher_id: 2 (张老师), semester: 2025秋
8  |— id: 102, name: 篮球, teacher_id: 2 (张老师), semester: 2025秋
9
10 student_progress (学时进度表) — 把学生和课程连起来
11 |— user_id: 1 (小明), course_id: 101 (足球), course_hours: 8, general_hours: 6
12 |— user_id: 1 (小明), course_id: 102 (篮球), course_hours: 4, general_hours:
13 10
14 sport_records (打卡记录表)
15 |— id: 5001, user_id: 1 (小明), course_id: 101, hours: 2, status: approved
16 |— id: 5002, user_id: 1 (小明), course_id: 101, hours: 1.5, status: pending
17 |— id: 5003, user_id: 1 (小明), course_id: null, hours: 3, status: approved,
18   type: general
19
20 reviews (审核记录表)
21 |— id: 8001, record_id: 5002, reviewer_id: 2 (张老师), decision: null
22 |— id: 8002, record_id: 5001, reviewer_id: 2 (张老师), decision: approve
23
24 notifications (通知表)
25 |— id: 9001, user_id: 1 (小明), title: 打卡审核通过, is_read: false
26 |— id: 9002, user_id: 1 (小明), title: 学时不足警告, is_read: true
```

关联是如何实现的：

- `sport_records.user_id = 1` → 这条打卡记录是小明的
- `sport_records.course_id = 101` → 这条记录属于足球课
- `reviews.record_id = 5002` → 这条审核记录对应 5002 号打卡
- `notifications.user_id = 1` → 这条通知是发给小明的

这种用 ID 互相引用的方式叫做**外键关联**（Foreign Key）。

6.3 宝塔面板里如何操作数据库

新增一条数据（比如添加一个新学生）：

代码块

```
1  -- 宝塔 → 数据库 → phpMyAdmin → SQL → 输入 → 执行
2
3  INSERT INTO users (username, password_hash, role, display_name)
4  VALUES ('xiaoming2025', 'hashed_password_here', 'student', '小明');
```

查询数据（看看小明有多少学时）：

代码块

```
1  SELECT u.display_name, sp.course_hours, sp.general_hours
2  FROM student_progress sp
3  JOIN users u ON u.id = sp.user_id
4  WHERE u.username = 'xiaoming2025';
5  -- 结果: 小明 | 8 | 6 → 课程 8h + 通用 6h = 14h 总计
```

修改表结构（比如给 users 表加一列手机号）：

代码块

```
1  ALTER TABLE users ADD COLUMN phone VARCHAR(20) DEFAULT '' AFTER display_name;
```

> **重要**：如果你的后端代码里用到了新加的列，修改完数据库后还需要 `pm2 restart bnbu-api-v1` 重启后端。数据库变更和代码变更是两件事。

第七章：Web 端如何连接后端

7.1 Web 端的请求路径

BNBU Sports 的 Web 端跑在老师的浏览器里。核心代码在 `app.js`。来看看它怎么发请求：

代码块

```
1  // app.js 第 51 行
2  let state = {
3      apiUrl: '',    // ← 空字符串！为什么？
4  };
5
6  // app.js 第 254 行 - 获取 API 地址的函数
7  function getApiBase() {
8      return state.apiUrl || window.location.origin;
9      //      ↑                ↑
10     //  如果人为填了就用      如果没填，自动取"当前网页的协议+域名+端口"
11     //  填的地址                = http://123.207.5.70:96
12 }
13
14 // 发请求时（实际代码在 app.js 各处）：
15 const apiUrl = getApiBase() + '/api/sport/summary';
16 //                = 'http://123.207.5.70:96' + '/api/sport/summary'
17 //                = 'http://123.207.5.70:96/api/sport/summary'
18
19 fetch(apiUrl)
20     .then(res => res.json())
21     .then(data => {
22         // data 就是从 Node.js 返回的数据
23         console.log('总学时:', data.totalHours);
24     });
```

为什么 `apiBaseUrl` 是空字符串？

因为 Web 端和 API 在**同一个 Nginx 端口**下。当你打开 `http://123.207.5.70:96`，浏览器就认为这个域名+端口是它的"家"。你用 `fetch('/api/sport/summary')` 发请求，浏览器自动在 `/api/...` 前面拼上 `http://123.207.5.70:96`。

然后 Nginx 发现 `/api/` → 转发给 Node.js。完美，不需要配任何地址。

7.2 一个完整的 Web 端请求示例

老师在浏览器里点击"审核"按钮，approve 小明的打卡：

代码块

```
1 // 浏览器里运行的 app.js
2 async function approveReview(reviewId) {
3   const apiBase = getApiBase(); // = 'http://123.207.5.70:96'
4
5   const response = await fetch(
6     `${apiBase}/api/teacher/reviews/${reviewId}/decision`,
7     {
8       method: 'PUT',
9       headers: { 'Content-Type': 'application/json' },
10      body: JSON.stringify({ decision: 'approve', hours: 2 })
11    }
12  );
13
14  const result = await response.json();
15  // result = { ok: true, message: '审核通过' }
16 }
```

链路追踪：

代码块

```
1 浏览器 fetch → http://123.207.5.70:96/api/teacher/reviews/8002/decision
2                ↓
3  Nginx :96    → 匹配到 location /api/ → proxy_pass http://127.0.0.1:3001
4                ↓
5  Node :3001   → app.put('/api/teacher/reviews/:id/decision', handler)
```

```

6      → 解析 reviewId=8002, decision=approve
7      → UPDATE reviews SET decision='approve' WHERE id=8002
8      → UPDATE sport_records SET status='approved' WHERE id=5002
9      → UPDATE student_progress SET course_hours = course_hours + 2
10     → INSERT INTO notifications (user_id, title) VALUES (1, '打卡审核
      通过')
11     → 返回 {"ok":true}
12     ↓
13     Nginx      → 原样传回
14     ↓
15     浏览器      → result.ok === true → 弹窗"审核成功", 刷新列表

```

第八章：Android App 如何连接后端

8.1 Android 和 Web 端的本质区别

对比维度	Web 端	Android App
运行在哪里	浏览器	Android 手机操作系统
使用什么发请求	<code>fetch()</code> API (浏览器自带)	OkHttp 库 (Gradle 依赖)
地址怎么配	<code>apiBaseUrl: ""</code> 自动取当前域名	<code>DefaultBaseUrl = "http://123.207.5.70:96/api"</code> 硬编码
有没有跨域限制	有 (同源策略)	没有 (CORS 只存在于浏览器)
需要什么额外配置	不需要	需要服务器返回 CORS 头 + Android 允许 HTTP 明文

8.2 Android 的 API 客户端是怎么工作的

看看真实的 Android 代码（来自你们的项目）：

代码块

```
1  // 文件:
   BNBUSStudentAndroid/app/src/main/java/edu/bnbu/student/mvp/core/network/StudentA
   piClient.kt
2
3  data class StudentApiClient(
4      val baseUrl: String = DefaultBaseUrl,    // 服务器地址
5      val bearerToken: String? = null,         // 登录后的 token
6  ) {
7      companion object {
8          // ⚠ 这是硬编码在 App 里的服务器地址!
9          const val DefaultBaseUrl = "http://123.207.5.70:96/api"
10     }
11
12     // 构建请求
13     fun request(endpoint: StudentEndpoint, body: Any? = null):
   StudentApiRequest {
14         val url = "${baseUrl.trimEnd('/')}/${endpoint.path.trimStart('/')}"
15         // 例如: http://123.207.5.70:96/api + / + sport/summary
16         //      = http://123.207.5.70:96/api/sport/summary
17
18         return StudentApiRequest(
19             method = endpoint.method,
20             url = url,
21             headers = mapOf(
22                 "Accept" to "application/json",
23                 "Authorization" to "Bearer ${bearerToken ?: ""}"
24             ),
25             body = body
26         )
27     }
28
29     // 真正发 HTTP 请求
30     fun execute(request: StudentApiRequest): String {
31         val builder = Request.Builder().url(request.url)
32
33         // GET / POST / PUT
34         when (request.method) {
35             HttpMethod.GET -> builder.get()
36             HttpMethod.POST -> builder.post(body)
37             HttpMethod.PUT -> builder.put(body)
```

```
38         }
39
40         // 用 OkHttpClient 发送并获取响应
41         val response =
42             SharedHttpClient.instance.newCall(builder.build()).execute()
43         return response.body?.string() ?: throw IOException("Empty response")
44     }
```

代码块

```
1 // 文件: StudentEndpoint.kt - 定义所有可用的 API 路径
2
3 sealed class StudentEndpoint(val method: HttpMethod) {
4     data object Login : StudentEndpoint(HttpMethod.POST)
5     data object SportSummary : StudentEndpoint(HttpMethod.GET)
6     data object SportRecords : StudentEndpoint(HttpMethod.POST)
7     data object SportIdentity : StudentEndpoint(HttpMethod.GET)
8     data object Notifications : StudentEndpoint(HttpMethod.GET)
9
10    val path: String
11        get() = when (this) {
12            Login -> "/auth/login"
13            SportSummary -> "/sport/summary"
14            SportRecords -> "/sport/records"
15            SportIdentity -> "/sport/identity"
16            Notifications -> "/common/notifications"
17            // ...
18        }
19 }
```

8.3 Android 发请求的完整链路

代码块

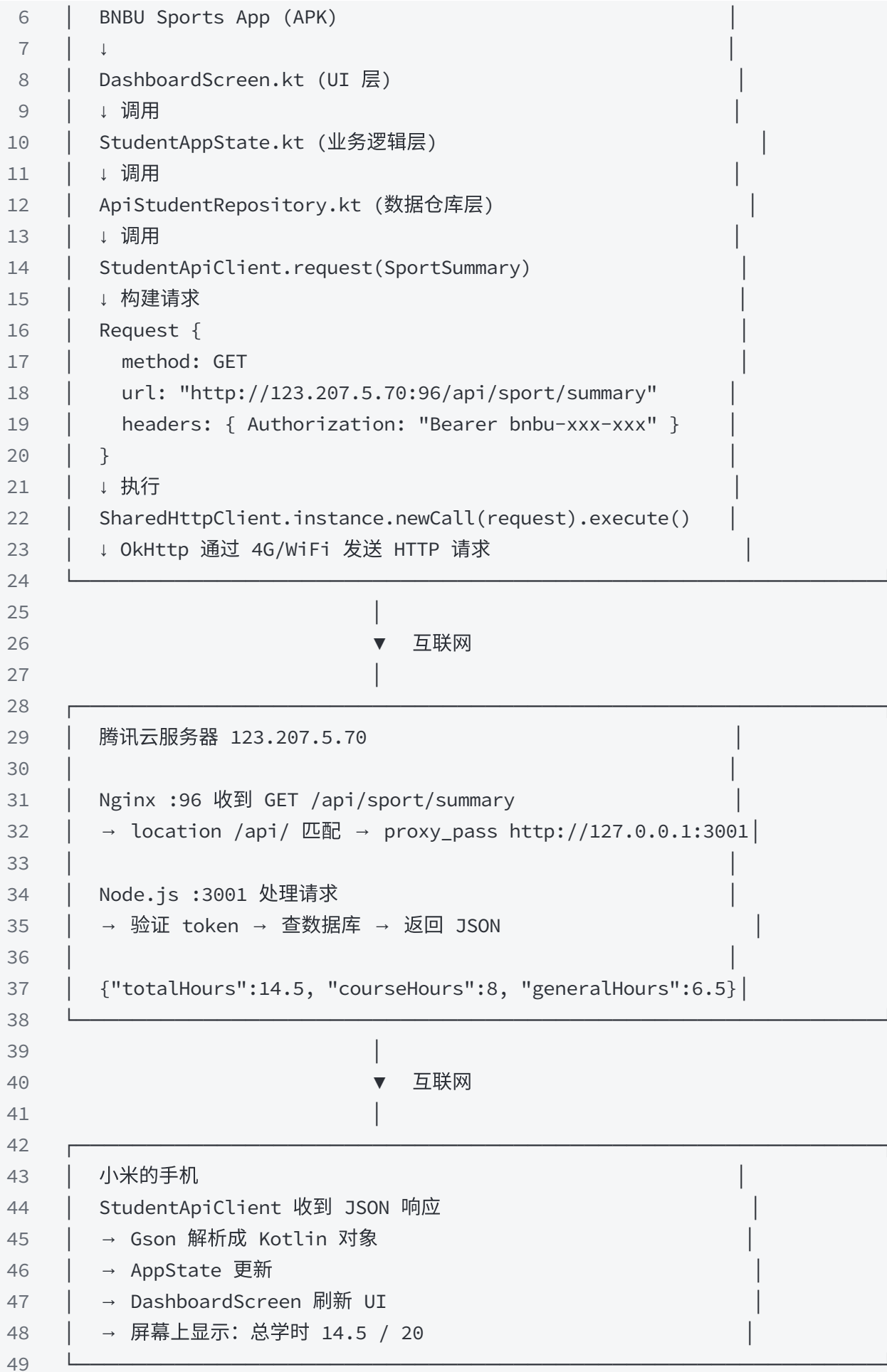
1 学生用手机打开 BNBU Sports App → 查看自己的学时:

2

3

4 | 小米的手机

5



8.4 Android 连接服务器需要的额外配置

① AndroidManifest.xml 需要允许 HTTP 明文

因为我们的服务器用的是 HTTP（不是 HTTPS），Android 默认会阻止明文流量：

代码块

```
1  <!-- AndroidManifest.xml -->
2  <application
3      android:usesCleartextTraffic="true"    ← 必须加这一行!
4      ...>
5  </application>
```

② 服务器需要配置 CORS 头

虽然 Android App 没有浏览器的同源限制，但 Node.js 的 Express 还是需要返回 CORS 头以允许 App 的请求。在 `server.js` 中已经有：

代码块

```
1  // server.js 中的 CORS 中间件
2  app.use((req, res, next) => {
3      res.header('Access-Control-Allow-Origin', '*');
4      res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE,
5  OPTIONS');
6      res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
7      if (req.method === 'OPTIONS') return res.sendStatus(200);
8      next();
9  });
```

③ App 需要处理网络权限

代码块


```
1 <!-- AndroidManifest.xml -->
2 <uses-permission android:name="android.permission.INTERNET" />
```

> ★ Insight

>

> **Android App 和 Web 端最大的不同**：App 是"纯粹的 HTTP 客户端"，它不关心你的前端 HTML/CSS。App 根本不用浏览器，它用 OkHttp（一个 Java HTTP 库）直接发 HTTP 请求给服务器。

>

> **这意味着**：即使你把 `/www/wwwroot/123.207.5.70_96/index.html` 删了，Android App 照样能正常工作。因为 App 只调 API，不调网页。

>

> **也意味着**：如果你改了 API 的返回格式（比如把字段名从 `total_hours` 改成 `totalHours`），你需要同时更新 Android 的数据模型（`StudentApiResponse.kt`）、iOS 的数据模型（`Models.swift`）和后端 `server.js` —— 三端都要改！

>

第九章：iOS App 如何连接后端

9.1 iOS 的架构（对标 Android）

iOS App 的结构和 Android 几乎一样，只是换了一套语言和框架：

层次	Android	iOS
语言	Kotlin	Swift
框架	Jetpack Compose	SwiftUI
HTTP 库	OkHttp	

		URLSession（系统自带，无需依赖）
JSON 解析	Gson	JSONDecoder（系统自带）
数据持久化	SharedPreferences	UserDefaults
API 客户端	StudentApiClient.kt	StudentAPIClient.swift t

9.2 iOS API 客户端的真实代码

代码块

```

1  // 文件: ios-app/BNBStudentApp/Core/StudentAPIClient.swift
2
3  struct StudentAPIClient {
4      let baseURL: URL
5      var token: String?
6
7      init(baseURL: URL = URL(string: "http://127.0.0.1:8080/api")!, token:
String? = nil) {
8          //                                ↑ 默认是本地开发地址，真机需要改成服务器地址
9          self.baseURL = baseURL
10         self.token = token
11     }
12
13     // 构建 URLRequest
14     func request(for endpoint: StudentEndpoint) -> URLRequest {
15         var request = URLRequest(url: baseURL.appending(path: endpoint.path))
16         request.httpMethod = endpoint.method.rawValue
17         request.setValue("application/json", forHTTPHeaderField: "Accept")
18         if let token {
19             request.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
20         }
21         return request
22     }
23 }
24
25 enum StudentEndpoint {
26     case login
27     case sportSummary
28     case sportRecords

```

```

29     case sportRecordDetail(id: String)
30     case supplementSportRecord(id: String)
31     case sportIdentity
32     case notifications
33     case markNotificationRead(id: String)
34
35     var path: String {
36         switch self {
37             case .login:           return "/auth/login"
38             case .sportSummary:    return "/sport/summary"
39             case .sportRecords:    return "/sport/records"
40             case .sportRecordDetail(let id): return "/sport/records/\"(id)\"
41             case .supplementSportRecord(let id): return "/sport/records/\"
42             case .sportIdentity:   return "/sport/identity"
43             case .notifications:   return "/common/notifications"
44             case .markNotificationRead(let id): return "/common/notifications/\"
45             case .markNotificationRead(let id): return "/common/notifications/\"
46             case .markNotificationRead(let id): return "/common/notifications/\"
47             case .markNotificationRead(let id): return "/common/notifications/\"

```

9.3 iOS 发请求的过程

代码块

```

1  iPhone 上的 BNB Sports App
2
3  ProfileView.swift (UI)
4      → AppState.swift (业务逻辑)
5          → StudentAPIClient.request(.sportSummary)
6              → URLSession.shared.data(for: request)
7                  → HTTP GET http://123.207.5.70:96/api/sport/summary
8                      → 服务器返回 JSON
9                          → JSONDecoder 解析成 Swift 结构体
10                             → AppState 更新
11                                 → SwiftUI 自动刷新 UI

```

9.4 iOS 上线前需要改的配置

目前 iOS App 的 `baseUrl` 默认是 `http://127.0.0.1:8080/api`（本地开发地址）。上线时应该改成：

代码块

```
1 // 上线前改成：
2 init(baseUrl: URL = URL(string: "http://123.207.5.70:96/api")!, token: String?
  = nil) {
```

iOS 的额外网络配置（Info.plist）：

代码块

```
1 <!-- 允许 HTTP 明文 (和 Android 一样的原因) -->
2 <key>NSAppTransportSecurity</key>
3 <dict>
4     <key>NSAllowsArbitraryLoads</key>
5     <true/>
6 </dict>
```

如果不加这个，iOS 会拒绝向 `http://`（非 HTTPS）地址发请求。

第十章：完整上传部署流程

现在我们以一个**真实的部署场景**来串联所有知识。假设你改了一个 Bug，需要把新版本上传到服务器。

场景：老师审核列表加载慢，你优化了 `app.js` 的查询逻辑

10.1 判断哪些文件需要上传

改了哪里	需要上传什么	不需要做什么
前端 <code>app.js</code>	上传 <code>app.js</code> 到 <code>/www/wwwroot/123.207.5.70_96/</code>	不用重启 PM2
前端 <code>styles.css</code>	上传 <code>styles.css</code> + 更新 <code>index.html</code> 的 <code>?v=</code> 版本号	不用重启 PM2
前端 <code>index.html</code>	上传 <code>index.html</code>	不用重启 PM2
后端 <code>server.js</code>	上传 <code>server.js</code> 到 <code>/www/wwwroot/bnbu-api-v1/</code>	要重启 PM2
后端 <code>package.json</code>	上传 <code>package.json</code> + 执行 <code>npm install</code>	要重启 PM2
新增数据库列	phpMyAdmin 执行 <code>ALTER TABLE</code>	可能要重启 PM2

口诀：

- 前端改 → 传文件 → 更新版本号 → 完事
- 后端改 → 传文件 → `pm2 restart` → 完事
- 数据库改 → phpMyAdmin → 可能要重启 PM2 → 完事

10.2 实操：上传前端文件

方法一：宝塔面板文件管理器（推荐，适合 1-4 个文件）

代码块

1. 宝塔面板 → 文件 → 导航到 `/www/wwwroot/123.207.5.70_96/`
2. 点击「上传」按钮

- 3 3. 选择本地的 `app.js` 文件
- 4 4. 勾选「覆盖已存在文件」
- 5 5. 确认上传
- 6 6. 同理上传 `styles.css`
- 7 7. 别忘了更新 `index.html` 里的 `?v=` 版本号!

方法二：宝塔面板的远程下载（适合大文件）

如果文件在 GitHub 上，直接贴原始 URL 让宝塔帮你下载。

方法三：SCP 命令行（适合自动化）

在你自己的电脑上（Windows PowerShell）：

代码块

```
1 scp video-showcase/app.js
  root@123.207.5.70:/www/wwwroot/123.207.5.70_100/app.js
2 scp video-showcase/styles.css
  root@123.207.5.70:/www/wwwroot/123.207.5.70_100/styles.css
```

10.3 实操：上传后端文件

代码块

```
1 1. 宝塔面板 → 文件 → 导航到 /www/wwwroot/bnbu-api-v1/
2 2. 上传新的 server.js（覆盖旧文件）
3 3. 宝塔面板 → 终端：
4   pm2 restart bnbu-api-v1
5 4. 验证：
6   curl http://127.0.0.1:3001/api/health
```

10.4 宝塔面板操作速查表

我想做什么	去哪	怎么操作
上传前端文件	文件 → /www/wwwroot/123.207.5 .70_96/	上传按钮 → 选文件 → 覆盖
上传后端文件	文件 → /www/wwwroot/bnbu-api- v1/	上传按钮 → 选文件 → 覆盖
改 Nginx 配置	网站 → 站点名 → 配置文件	编辑 → 保存 → 重载
看 API 是否正常	终端	curl http://127.0.0.1:3001/ api/health
看后端报错	终端	pm2 logs bnbu-api-v1
添加数据库列	数据库 → phpMyAdmin → SQL	输入 ALTER TABLE → 执行
看数据库内容	数据库 → phpMyAdmin → 表名	点击表名 → 浏览
上传 APK	文件 → /www/wwwroot/123.207.5 .70_96/	上传 app-debug.apk
看谁在访问	网站 → 站点名 → 日志	查看访问日志

第十一章：日常维护与故障排查

11.1 故障排查五步法

当用户反馈"系统用不了了"，按以下顺序排查：

代码块

```
1  第一步: curl http://127.0.0.1:3001/api/health
2      |—— 返回 JSON → Node.js 正常, 跳到第三步
3      |—— 没响应 → 去第二步
4
5  第二步: pm2 list
6      |—— bnbu-api-v1 状态是 online → 可能是端口问题, 跳到第三步
7      |—— bnbu-api-v1 状态是 stopped → pm2 restart bnbu-api-v1, 再试第一步
8
9  第三步: curl http://123.207.5.70:96/api/health
10     |—— 返回 JSON → Nginx 正常, API 正常, 问题在前端
11     |   |—— 检查前端文件是否完整上传 → 浏览器 F12 看 Console 报错
12     |—— 返回 502 → Nginx 活着但连不上 Node.js
13         |—— 检查 Node.js 端口: netstat -tlnp | grep 3001
14
15 第四步: 如果 API 正常但前端页面空白
16     → 浏览器 F12 → Network 标签 → 看 app.js 是否加载成功
17     → 看 app.js 是否有 JavaScript 报错
18     → 可能是 app.js 不完整 (上传中断?)
19
20 第五步: 如果 API 返回数据库错误
21     → pm2 logs bnbu-api-v1 → 看具体错误
22     → 可能是数据库表结构改了但代码没跟上
23     → 去 phpMyAdmin 检查表结构
```

11.2 常见问题速查

症状	可能原因	解决
访问网页白屏	app.js 没上传完整	重新上传 app.js , 检查文件大小
API 返回 502 Bad Gateway	Node.js 没启动	pm2 restart bnbu-api-v1
API 返回 404 Not Found	路径写错了 或 路由没定义	检查 URL 拼写, 看 server.js 里有没有对应路由

API 返回 500 Internal Server Error	后端代码报错	<code>pm2 logs bnbu-api-v1</code> 看具体错误
改了 CSS 但页面没变化	浏览器缓存	Ctrl+Shift+R 强制刷新；检查 <code>index.html</code> 的 <code>?v=</code> 是否更新
数据库连接失败	MySQL 没启动	<code>systemctl start mysql</code> 或宝塔重启 MySQL
Android App 连不上	网络或防火墙	检查手机网络能否 ping 通 <code>123.207.5.70</code>
<code>pm2: command not found</code>	宝塔终端 PATH 问题	创建软链接（见第二章终端一节）
Node.js 突然挂掉	内存不足或未捕获异常	<code>pm2 logs bnbu-api-v1</code> 看挂之前的最后几行

11.3 BNB Sports 项目部署清单（最终对照表）

文件	本地路径	服务器路径	改了之后做什么
前端首页	<code>web-app/index.html</code>	<code>/www/wwwroot/123.207.5.70_96/index.html</code>	上传即可
前端逻辑	<code>web-app/app.js</code>	<code>/www/wwwroot/123.207.5.70_96/app.js</code>	上传 + 更新 <code>index.html</code> 的 <code>?v=</code>
前端样式1	<code>web-app/styles-campus-blue.css</code>	<code>/www/wwwroot/123.207.5.70_96/styles-campus-blue.css</code>	上传 + 更新 <code>?v=</code>
前端样式2	<code>web-app/styles.css</code>	<code>/www/wwwroot/123.207.5.70_96/styles.css</code>	上传 + 更新 <code>?v=</code>
后端核心	<code>backend-handoff/server.js</code>	<code>/www/wwwroot/bnbu-api-v1/server.js</code>	上传 + <code>pm2 restart bnbu-api-v1</code>
依赖声明	<code>backend-handoff/package.json</code>	<code>/www/wwwroot/bnbu-api-v1/package.json</code>	上传 + <code>npm install</code> + <code>pm2 restart</code>

	n		
数据库	backend-handoff/schema.sql	phpMyAdmin 执行 SQL	在 phpMyAdmin 里执行
安卓APK	app-debug.apk	/www/wwwroot/123.207.5.70_96/bnbu-student.apk	上传即可
iOS App	Xcode 项目	App Store (未来)	目前本地开发

11.4 Video Showcase 部署清单

文件	本地路径	服务器路径	改了之后做什么
首页	video-showcase/index.html	/www/wwwroot/123.207.5.70_100/index.html	上传即可
JS	video-showcase/app.js	/www/wwwroot/123.207.5.70_100/app.js	上传 + 更新 index.html 的 ?v=
样式	video-showcase/styles.css	/www/wwwroot/123.207.5.70_100/styles.css	上传 + 更新 ?v=
后端	video-showcase/server.js	/www/wwwroot/video-showcase-api/server.js	上传 + pm2 restart vshow-api
认证	video-showcase/auth.js	/www/wwwroot/video-showcase-api/auth.js	上传 + pm2 restart vshow-api

 实践篇：自己动手搭建一个网站（安全区域）

> **读十遍不如做一遍**。这一节教你用宝塔面板**新建一个完全属于你自己的网站**，从零搭建一个带前端+后端+数据库的完整 Web 应用。

>

> 我们用的是服务器上一个**新的端口**，与生产环境的 `:96` 和 `:100` 完全隔离，放心操作。

实践目标

最终你会拥有一个跑在 `http://123.207.5.70:3333` 的小网站，它：

- 前端：一个能输入留言并显示所有留言的网页
- 后端：一个 Node.js API，接收留言并存到数据库
- 数据库：一张 MySQL 表存储留言
- Nginx 反向代理：把前端和 API 拴在一起

整个流程就是一次完整的"从零到全栈部署"体验。

第〇步：在服务器上开一个安全新端口

代码块

1. 宝塔面板 → 网站 → 添加站点
2. 填写：
 - 域名：123.207.5.70:3333
 - （如果你有域名比如 test.bnbu.com，填域名更好，这里我们用 IP 最简单）
 - 根目录：/www/wwwroot/123.207.5.70_3333/
 - PHP版本：纯静态（我们不用 PHP）
3. 点击「提交」

这时宝塔自动做了几件事：

- 创建了文件夹 `/www/wwwroot/123.207.5.70_3333/`
- 在 Nginx 里注册了端口 3333 的站点
- 自动生成了一份基础 Nginx 配置

验证：在浏览器打开 `http://123.207.5.70:3333`，应该看到"站点创建成功"的默认页面。

> 想想第二章的知识：你刚刚做的是"在餐馆里加了一间新包厢"——新的端口 = 新的入口。

第一步：搭建前端（用纯 HTML 写一个留言板）

在宝塔面板 → 文件 → 导航到 `/www/wwwroot/123.207.5.70_3333/`，删除默认的 `index.html`。

点击「新建文件」→ 文件名 `index.html`，粘贴以下内容：

代码块

```
1  <!DOCTYPE html>
2  <html lang="zh-CN">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>我的第一个全栈网站</title>
7      <style>
8          * { margin: 0; padding: 0; box-sizing: border-box; }
9          body {
10              font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-
11              serif;
12              background: #f0f4f8; color: #333;
13              display: flex; justify-content: center; padding-top: 60px;
14          }
15          .container { width: 100%; max-width: 600px; padding: 0 20px; }
16          h1 { text-align: center; margin-bottom: 32px; color: #2d3748; font-
17          size: 28px; }
18          .card {
19              background: white; border-radius: 8px; padding: 24px;
20              box-shadow: 0 2px 8px rgba(0,0,0,0.08); margin-bottom: 24px;
21          }
22          h2 { font-size: 18px; margin-bottom: 16px; color: #4a5568; }
23          textarea {
24              width: 100%; height: 80px; padding: 12px; border: 1px solid
25              #e2e8f0;
```

```

23         border-radius: 6px; font-size: 14px; resize: vertical; font-
family: inherit;
24     }
25     textarea:focus { outline: none; border-color: #4299e1; box-shadow: 0 0
0 3px rgba(66,153,225,0.15); }
26     input[type="text"] {
27         width: 100%; padding: 10px 12px; border: 1px solid #e2e8f0;
28         border-radius: 6px; font-size: 14px; margin-bottom: 12px; font-
family: inherit;
29     }
30     input[type="text"]:focus { outline: none; border-color: #4299e1; box-
shadow: 0 0 0 3px rgba(66,153,225,0.15); }
31     button {
32         background: #4299e1; color: white; border: none; padding: 10px
24px;
33         border-radius: 6px; font-size: 14px; cursor: pointer; font-weight:
600;
34         margin-top: 12px;
35     }
36     button:hover { background: #3182ce; }
37     button:active { background: #2b6cb0; transform: scale(0.98); }
38     .msg-item {
39         padding: 14px 0; border-bottom: 1px solid #edf2f7;
40     }
41     .msg-item:last-child { border-bottom: none; }
42     .msg-author { font-weight: 700; color: #2d3748; margin-bottom: 4px; }
43     .msg-text { color: #4a5568; font-size: 14px; line-height: 1.6; }
44     .msg-time { color: #a0aec0; font-size: 12px; margin-top: 6px; }
45     .loading { text-align: center; color: #a0aec0; padding: 20px; }
46     .empty { text-align: center; color: #a0aec0; padding: 40px 0; }
47     .error { color: #e53e3e; font-size: 14px; margin-top: 8px; display:
none; }
48     .success { color: #38a169; font-size: 14px; margin-top: 8px; display:
none; }
49     </style>
50 </head>
51 <body>
52     <div class="container">
53         <h1>👋 我的留言板</h1>
54
55         <!-- 发留言的表单 -->
56         <div class="card">
57             <h2>👉 写留言</h2>
58             <input type="text" id="author" placeholder="你的名字"
maxLength="30" />
59             <textarea id="content" placeholder="想说点什么..." maxLength="500">
</textarea>

```

```

60         <div class="error" id="submitError"></div>
61         <div class="success" id="submitSuccess"></div>
62         <button onclick="submitMessage()">👉 发布留言</button>
63     </div>
64
65     <!-- 留言列表 -->
66     <div class="card">
67         <h2>💬 所有留言</h2>
68         <div id="messageList">
69             <div class="loading">加载中...</div>
70         </div>
71     </div>
72 </div>
73
74 <script>
75     // 因为前端和 API 在同一个端口下, 用相对路径即可
76     const API_URL = '/api';
77
78     // 页面加载时获取所有留言
79     window.onload = () => loadMessages();
80
81     async function loadMessages() {
82         const listEl = document.getElementById('messageList');
83         try {
84             const resp = await fetch(`${API_URL}/messages`);
85             const data = await resp.json();
86             if (!data.ok) throw new Error(data.error || '未知错误');
87             const messages = data.messages || [];
88
89             if (messages.length === 0) {
90                 listEl.innerHTML = '<div class="empty">还没有留言, 来做第一个
91 发言的人吧! 🙋</div>';
92                 return;
93             }
94
95             listEl.innerHTML = messages.map(m => `
96                 <div class="msg-item">
97                     <div class="msg-author">${escapeHtml(m.author)}</div>
98                     <div class="msg-text">${escapeHtml(m.content)}</div>
99                     <div class="msg-time">${m.created_at}</div>
100                 </div>
101             `).join('');
102         } catch (err) {
103             listEl.innerHTML = `<div class="error" style="display:block;">
104 加载失败: ${escapeHtml(err.message)}</div>`;
105         }
106     }

```

```

105
106     async function submitMessage() {
107         const authorEl = document.getElementById('author');
108         const contentEl = document.getElementById('content');
109         const errEl = document.getElementById('submitError');
110         const succEl = document.getElementById('submitSuccess');
111
112         errEl.style.display = 'none';
113         succEl.style.display = 'none';
114
115         const author = authorEl.value.trim();
116         const content = contentEl.value.trim();
117
118         if (!author) { errEl.textContent = '请输入你的名字';
errEl.style.display = 'block'; return; }
119         if (!content) { errEl.textContent = '请输入留言内容';
errEl.style.display = 'block'; return; }
120
121         try {
122             const resp = await fetch(`${API_URL}/messages`, {
123                 method: 'POST',
124                 headers: { 'Content-Type': 'application/json' },
125                 body: JSON.stringify({ author, content })
126             });
127             const data = await resp.json();
128             if (!data.ok) throw new Error(data.error || '发布失败');
129
130             authorEl.value = '';
131             contentEl.value = '';
132             succEl.textContent = '✅ 留言发布成功! ';
133             succEl.style.display = 'block';
134             setTimeout(() => { succEl.style.display = 'none'; }, 3000);
135             loadMessages();
136         } catch (err) {
137             errEl.textContent = '❌ ' + escapeHtml(err.message);
138             errEl.style.display = 'block';
139         }
140     }
141
142     function escapeHtml(text) {
143         const div = document.createElement('div');
144         div.textContent = text;
145         return div.innerHTML;
146     }
147 </script>
148 </body>

```

> **想想第三章的知识：**这个 HTML 文件就是一个**静态文件**。它存在 `/www/wwwroot/123.207.5.70_3333/` 里，Nginx 会直接把它发给浏览器，浏览器执行里面的 JavaScript。里面的 `fetch('/api/messages')` 就是前端向后端发请求的指令。

第二步：创建数据库

现在需要一个数据库来存储留言。

代码块

- 1 1. 宝塔面板 → 数据库 → 添加数据库
- 2 2. 填写：
- 3 数据库名：mydb_practice
- 4 用户名： mydb_practice
- 5 密码： 自己设一个强密码（记下来，等下要用）
- 6 访问权限：本地服务器
- 7 3. 点击「提交」
- 8 4. 点击 mydb_practice 旁边的「管理」按钮 → 进入 phpMyAdmin
- 9 5. 在 phpMyAdmin 里点「SQL」标签 → 粘贴以下建表语句 → 执行：

代码块

```
1 CREATE TABLE messages (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     author VARCHAR(50) NOT NULL COMMENT '留言者名字',  
4     content TEXT NOT NULL COMMENT '留言内容',  
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT '留言时间'  
6 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='留言板表';
```

执行后，你会在左侧看到 `messages` 表出现在 `mydb_practice` 数据库下。

> 想想第六章的知识：你刚刚创建了一张表。`id` 是主键（每条留言的唯一编号），`author` 是留言者，`content` 是内容，`created_at` 自动记录时间。

第三步：编写后端 API (Node.js server.js)

切换到宝塔面板 → 文件。

先在 `/www/wwwroot/` 下新建一个文件夹作为你的 API 目录：

代码块

1. 导航到 `/www/wwwroot/`
2. 新建文件夹 → 命名为 `mydb-practice-api`

然后进入这个文件夹，新建文件 `server.js`，粘贴以下内容：

代码块

```
1  // 留言板 API 后端
2  const express = require('express');
3  const mysql = require('mysql2/promise');
4
5  const app = express();
6  app.use(express.json());
7
8  // — CORS (允许来自任何地方的跨域请求, 方便开发调试) —
9  app.use((req, res, next) => {
10     res.header('Access-Control-Allow-Origin', '*');
11     res.header('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
12     res.header('Access-Control-Allow-Headers', 'Content-Type');
13     if (req.method === 'OPTIONS') return res.sendStatus(200);
14     next();
15 });
16
17 // — 数据库连接池 (把密码换成你刚才设的!) —
18 const pool = mysql.createPool({
19     host: '127.0.0.1',
20     user: 'mydb_practice',
```

```
21     password: '你设的密码',    // ← 改这里!
22     database: 'mydb_practice',
23     waitForConnections: true,
24     connectionLimit: 5,
25     charset: 'utf8mb4',
26 });
27
28 // — 健康检查 —
29 app.get('/api/health', async (req, res) => {
30     try {
31         const [rows] = await pool.query('SELECT 1 AS alive');
32         res.json({ ok: true, db: rows[0].alive === 1 });
33     } catch (err) {
34         res.status(500).json({ ok: false, error: err.message });
35     }
36 });
37
38 // — 获取所有留言 —
39 app.get('/api/messages', async (req, res) => {
40     try {
41         const [rows] = await pool.query(
42             'SELECT id, author, content, created_at FROM messages ORDER BY id
DESC LIMIT 100'
43         );
44         res.json({ ok: true, messages: rows });
45     } catch (err) {
46         res.status(500).json({ ok: false, error: err.message });
47     }
48 });
49
50 // — 发布新留言 —
51 app.post('/api/messages', async (req, res) => {
52     const { author, content } = req.body;
53
54     // 校验
55     if (!author || !author.trim()) {
56         return res.status(400).json({ ok: false, error: '请填写名字' });
57     }
58     if (!content || !content.trim()) {
59         return res.status(400).json({ ok: false, error: '请填写留言内容' });
60     }
61     if (author.length > 50) {
62         return res.status(400).json({ ok: false, error: '名字太长 (最多50个字)'
});
63     }
64     if (content.length > 500) {
```

```

65         return res.status(400).json({ ok: false, error: '留言太长（最多500个字） '
66     });
67
68     try {
69         const [result] = await pool.query(
70             'INSERT INTO messages (author, content) VALUES (?, ?)',
71             [author.trim(), content.trim()]
72         );
73         res.json({ ok: true, id: result.insertId, message: '留言成功! ' });
74     } catch (err) {
75         res.status(500).json({ ok: false, error: err.message });
76     }
77 });
78
79 // — 启动服务（用一个新的端口，不跟生产环境冲突） —
80 const PORT = 3334;
81 app.listen(PORT, '127.0.0.1', () => {
82     console.log(`留言板 API 已启动: http://127.0.0.1:${PORT}`);
83     console.log(`健康检查: curl http://127.0.0.1:' + PORT + '/api/health');
84 });

```

> 想想第四章的知识：这个 `server.js` 就是一个后端。它监听 3334 端口（127.0.0.1 表示只允许本地访问），连接 MySQL 数据库，定义了两个 API 端点： `GET /api/messages`（查留言）和 `POST /api/messages`（发留言）。

新建文件 `package.json`，粘贴：

代码块

```

1  {
2      "name": "mydb-practice-api",
3      "version": "1.0.0",
4      "private": true,
5      "scripts": {
6          "start": "node server.js"
7      },
8      "dependencies": {
9          "express": "^4.18.2",
10         "mysql2": "^3.9.0"
11     }

```

```
12 }
```

> 这就是依赖声明。告诉 Node.js：我这个项目需要 `express` 和 `mysql2` 两个库。

第四步：安装依赖并启动后端

打开宝塔面板 → 终端：

代码块

```
1  # ① 进到你的 API 目录
2  cd /www/wwwroot/mydb-practice-api
3
4  # ② 安装依赖（需要 PATH 中有 node 和 npm，如果 command not found，看第二章的软链接教程）
5  npm install
6  # 看到 "added XX packages" 就成功了
7
8  # ③ 直接运行一下，看看有没有报错
9  node server.js
10 # 如果看到 "留言板 API 已启动: http://127.0.0.1:3334" → 成功!
11 # Ctrl+C 退出（我们一会儿用 PM2 来管理它）
12
13 # ④ 用 PM2 启动（挂了会自动重启）
14 pm2 start server.js --name mydb-practice
15 # 看到 status: online 就 OK
16
17 # ⑤ 让 PM2 记住这个进程（服务器重启后自动恢复）
18 pm2 save
19
20 # ⑥ 验证：直接在服务器上请求
21 curl http://127.0.0.1:3334/api/health
22 # 预期输出: {"ok":true,"db":true}
```

> ★ Insight

>

> `npm install` 到底是什么？你的 `server.js` 里写了 `require('express')` 和 `require('mysql2')`，这两行代码告诉 Node.js "我需要用到 express 和 mysql2"。但这两个库不是 Node.js 自带的——npm 会去远程仓库下载它们的代码，放到 `node_modules/` 文件夹里。所以每次新增依赖（改 `package.json`），都必须跑一次 `npm install`。

>

> **PM2 是什么？** `node server.js` 会让你看到日志，但一关终端就停了。`pm2 start` 让进程在后台持续运行，进程挂了会自动重启，服务器重启后也能自动恢复（如果你执行过 `pm2 save`）。它是生产环境的必需品。

>

第五步：配置 Nginx 反向代理（拴住前后端）

现在浏览器只能访问 `http://123.207.5.70:3333` 看到前端页面，但 `fetch('/api/messages')` 发出去请求会返回 404——因为 Nginx 还不知道要把 `/api/` 请求转发给后端的 3334 端口。

代码块

1. 宝塔面板 → 网站 → 站点 123.207.5.70:3333 → 配置文件
2. 找到 `location / { ... }` 块（大概在第 10-15 行）
3. 在它的 `}` 后面，加上下面的反向代理配置：

代码块

```
1  # — API 反向代理: /api/* 转发给 Node.js —
2  location /api/ {
3      proxy_pass http://127.0.0.1:3334;
4      proxy_http_version 1.1;
5      proxy_set_header Host $host;
6      proxy_set_header X-Real-IP $remote_addr;
7      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
8      proxy_read_timeout 10s;
9      proxy_connect_timeout 3s;
10 }
```

完整配置大概长这样：

代码块

```
1  server {
2      listen 3333;
3      server_name 123.207.5.70;
4      root /www/wwwroot/123.207.5.70_3333;
5      index index.html;
6
7      # 静态文件
8      location / {
9          try_files $uri $uri/ /index.html;
10     }
11
12     # API 反向代理 ← 你刚刚加的这一段
13     location /api/ {
14         proxy_pass http://127.0.0.1:3334;
15         proxy_http_version 1.1;
16         proxy_set_header Host $host;
17         proxy_set_header X-Real-IP $remote_addr;
18         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
19         proxy_read_timeout 10s;
20         proxy_connect_timeout 3s;
21     }
22 }
```

保存 → 点击「重载配置」（或者去终端执行 `nginx -s reload`）。

> 想想第五章的知识：你刚刚写的就是反向代理配置。`location /api/` 告诉 Nginx："以 `/api/` 开头的请求不要找文件，全部转给 `http://127.0.0.1:3334` 的 Node.js 处理。"这就是前端和后端之间的"接线端子"。

第六步：验证！完整的链路上线测试

打开浏览器，访问 `http://123.207.5.70:3333`：

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

 我的留言板

 写留言

小明

← 输入名字

大家好，这是我的第一条留言！

← 输入内容

 发布留言

 所有留言

小明

大家好，这是我的第一条留言！

2026-06-29 15:30:01

测试清单：

- ☐ 页面正常加载（Nginx 把 index.html 发过来了）
- ☐ 输入名字和留言，点击发布，页面显示"留言发布成功"
- ☐ 刷新页面，刚才发的留言还在（说明数据存到了 MySQL）
- ☐ 用另一台设备打开 `http://123.207.5.70:3333`，也能看到同样的留言
- ☐ 在宝塔终端执行 `curl http://127.0.0.1:3334/api/health`，返回 `{"ok":true,"db":true}`

如果某一步失败了，请回顾第十一章的故障排查五步法。

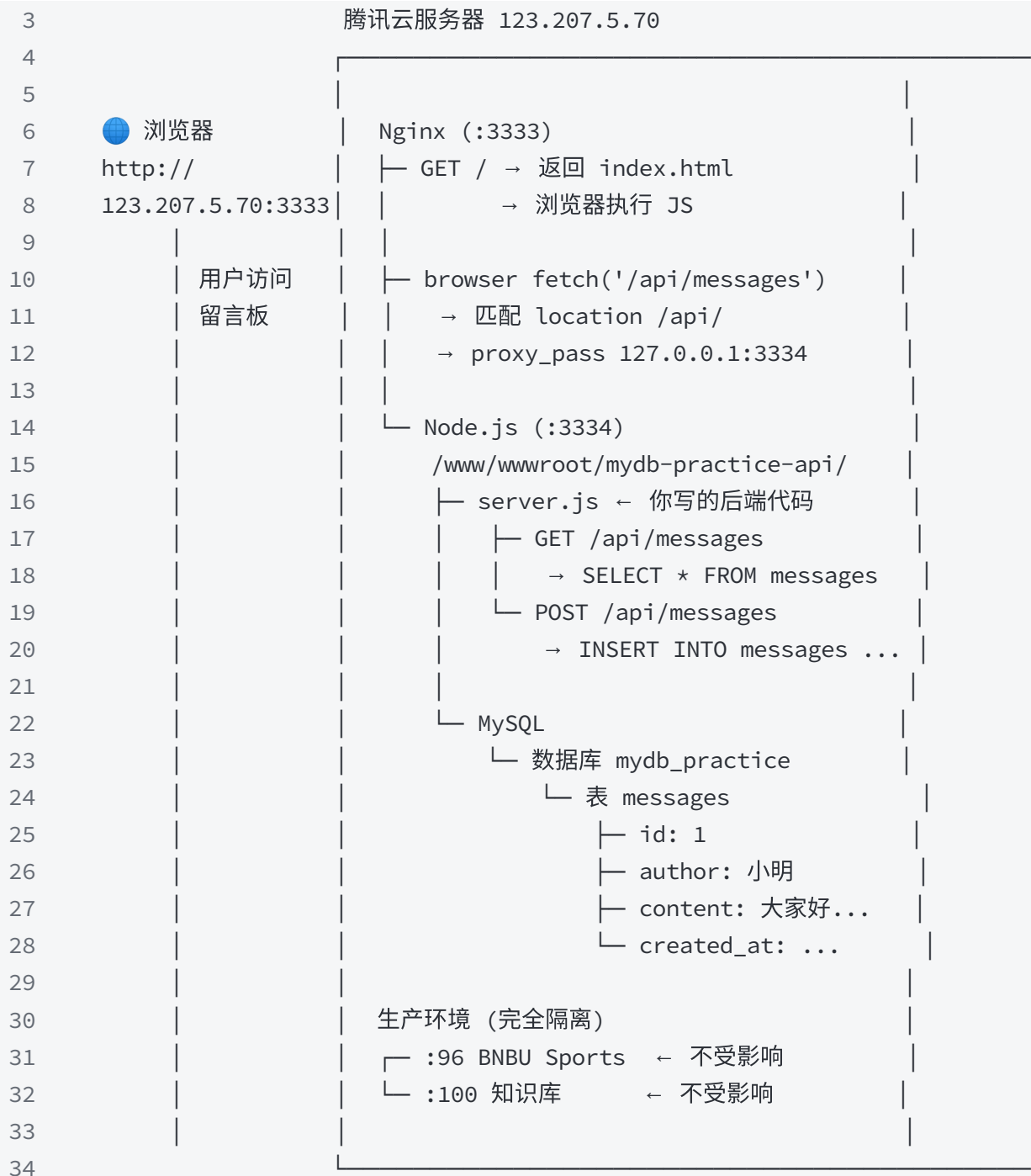
第七步：理解你刚刚做了什么

代码块

1

2

你刚刚搭建的完整全栈架构：



你亲手搭建的链路：

1. 浏览器访问 `:3333` → Nginx 返回 HTML → 浏览器渲染页面
2. 你在输入框打字 → 点击"发布" → JS 执行 `fetch('/api/messages', { method: 'POST', ... })`
3. 请求到达 Nginx `:3333` → 匹配 `/api/` → 转发给 Node.js `:3334`
4. Node.js `server.js` 收到请求 → 校验输入 → `INSERT INTO messages` → 存进 MySQL
5. Node.js 返回 `{"ok": true}` → Nginx 传给浏览器 → JS 刷新留言列表

6. JS 执行 `fetch('/api/messages')` → 同样的链路 → `SELECT * FROM messages` → 显示所有留言

这就是全栈开发的全部本质。你用的端口和数据库都是全新的，跟生产环境 `:96` 和 `:100` 毫无关系。

实践后清理（可选）

如果你以后不想让这个练习网站继续运行：

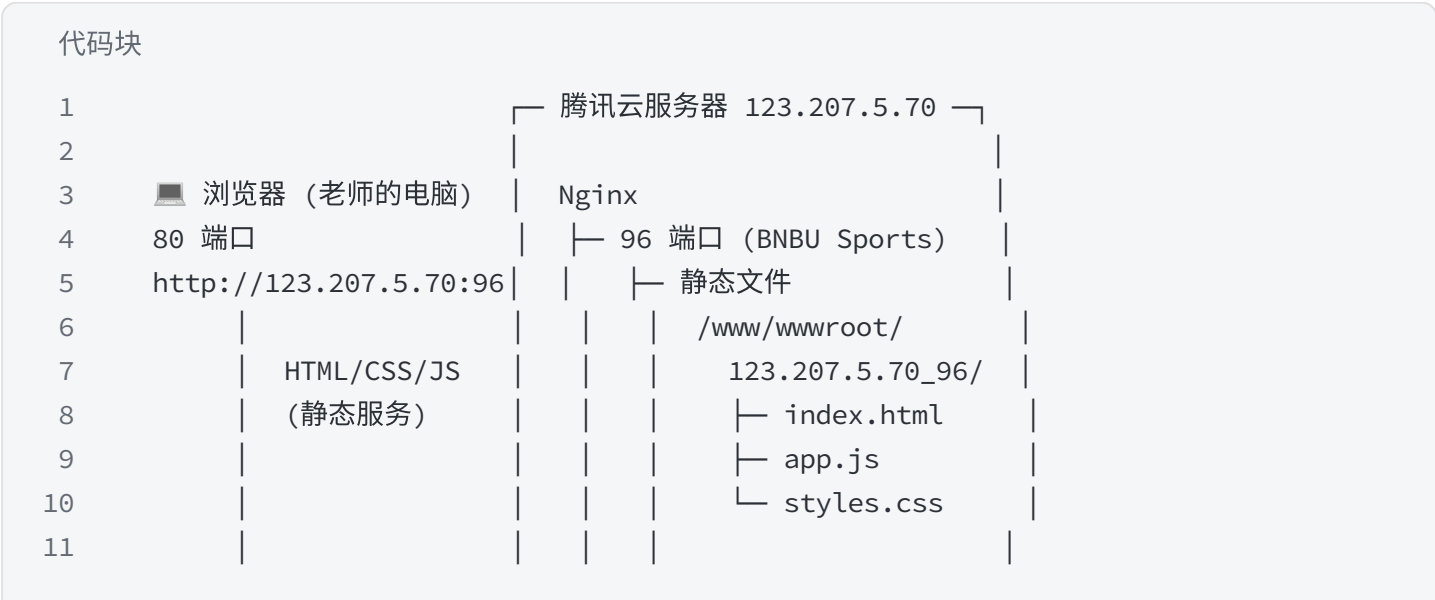
代码块

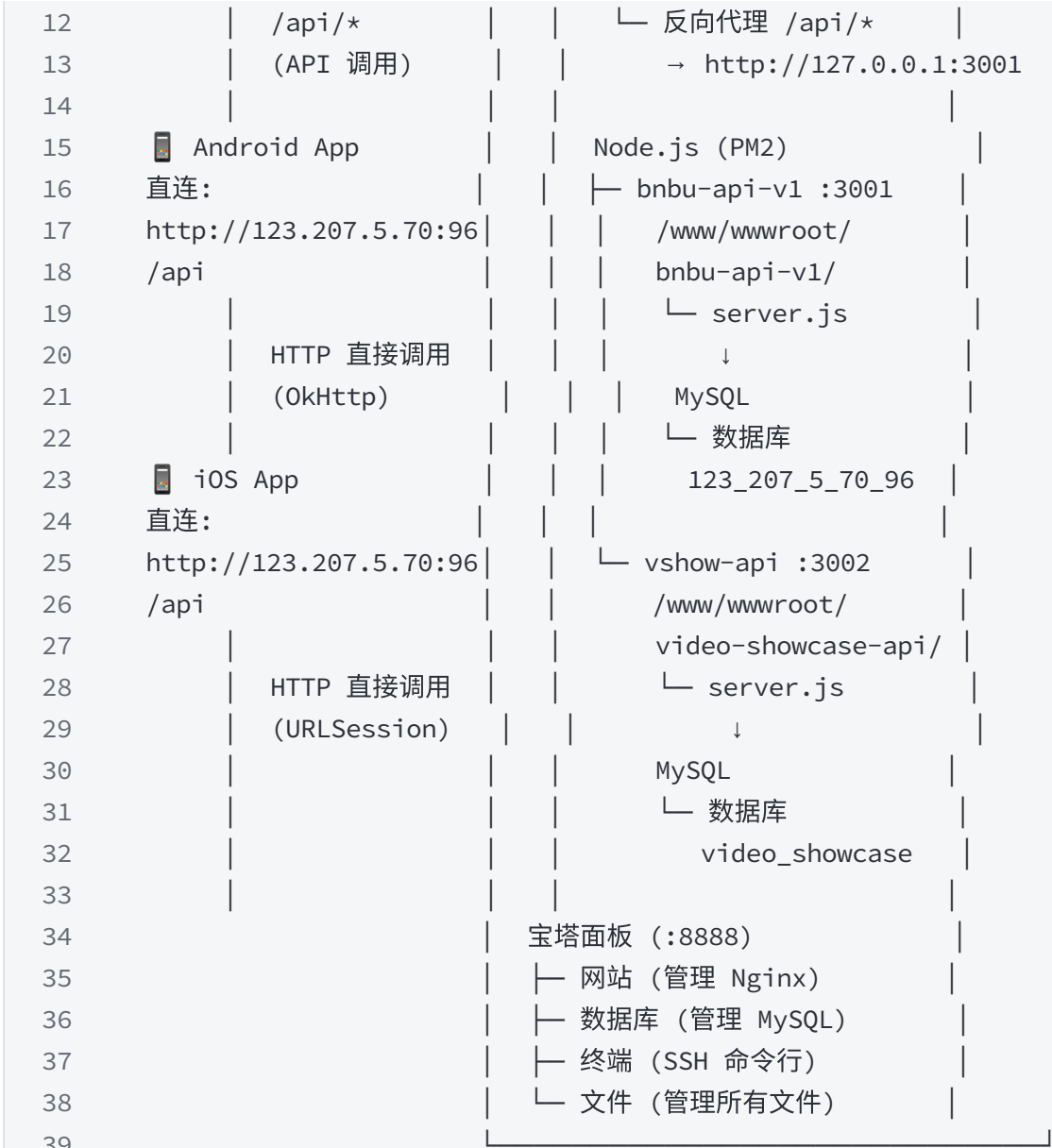
```
1 # 宝塔终端：
2 pm2 delete mydb-practice # 停止并删除 PM2 进程
3 # 宝塔文件：
4 # 删除 /www/wwwroot/123.207.5.70_3333/ 和 /www/wwwroot/mydb-practice-api/
5 # 宝塔网站：
6 # 删除站点 123.207.5.70:3333
7 # 宝塔数据库：
8 # 删除数据库 mydb_practice
```

术语	通俗解释	在本项目中的实例
IP 地址	服务器在互联网上的门牌号	123.207.5.70
端口	同一台服务器上的不同"柜台"	96 = BNBU Sports 前端, 3001 = API
Nginx	负责"接待客人+传菜"的 Web 服务器	给用户返回 HTML/JS/CSS, 转发 API
Node.js	负责"做菜"的后端运行时	执行 <code>server.js</code> , 连接数据库
Express	Node.js 里最流行的 Web 框架	<code>app.get('/api/...', handler)</code>
MySQL	关系型数据库	存学生、课程、打卡、成绩

phpMyAdmin	MySQL 的网页版管理工具	在宝塔面板里操作数据库
PM2	Node.js 进程守护者	保证 <code>server.js</code> 挂了自动重启
SPA	单页应用，用 JS 动态生成页面	<code>index.html</code> 很空，全靠 <code>app.js</code> 渲染
API	应用程序接口（前后端交流的语言）	<code>GET /api/sport/summary</code>
JSON	前后端数据传输的格式	<code>{"totalHours": 12.5}</code>
反向代理	Nginx 帮用户把请求转发给后端	<code>location /api/ { proxy_pass ... }</code>
CORS	跨域资源共享（浏览器安全策略）	服务器返回 <code>Access-Control-Allow-Origin: *</code>
静态文件	不需要服务器计算的纯文件	<code>index.html</code> , <code>app.js</code> , <code>styles.css</code>
Token	用户登录后拿到的"通行证"	<code>bnbu-xxxx-xxxx-xxxx</code>
Git	版本控制工具	管理代码历史，多人协作
SSH	远程登录服务器的安全协议	<code>ssh root@123.207.5.70</code>

附录B：一张图总结全部关系





> 本文档基于 BNBU Sports 项目真实架构编写。所有 IP、端口、密码、目录路径均为实际生产环境的配置。

>

> 最后更新：2026-06-29

>

> 相关问题请参考：

> - [video-showcase/DEPLOY.md](#) — Video Showcase 部署指南

> - [video-showcase/DEPLOY_ZEN.md](#) — 精简版部署清单

> - `Baota server configure.md` — BNB Sports V1 完整服务器配置指南

> - `CLAUDE.md` — 所有项目配置的总索引